卫星导航算法与程序设计 1

成果总结报告

姓名：wwz

2025 年 3 月 31 日 至 2025 年 7 月 30 日

目录

一、 程序设计概述 1
1.1 软件功能 1
1.2 软件接口 2
1.2.1 硬件对接接口 2
1.2.2 数据输入接口 2
1.2.3 数据输出接口 3
1.3 编程语言 3
1.4 完成情况 3
二、 算法与程序设计 4
2.1 总体设计 4
2.2 时间转换 6
2.2.1 计时法与结构体定义 6
2.2.2. 通用时转换为简化儒略日 7
2.2.3 儒略日转换为通用时 8
2.2.4 简化儒略日转换为 GPS 时 9
2.2.5 GPS 时转换为简化儒略日 9
2.2.6 通用时与 GPS 时的相互转换 10
2.3 坐标转换 11
2.3.1 大地坐标转换到笛卡尔坐标 11
2.3.2 笛卡尔坐标转换到大地坐标 12
2.3.3 卫星高度角和方位角计算 12
2.3.4 定位误差计算 13
2.4 NovAtel OEM719 数据解码 14
2.4.1 解码主函数 16
2.4.2 观测数据解码函数 19
2.4.3 GPS 星历解码函数 21
2.4.4 BDS 星历解码函数 23
2.4.5 定位结果解码函数 24
2.5 卫星位置计算 25
2.5.1 卫星位置计算主函数 26
2.5.2 卫星钟差计算函数 26
2.5.3 GPS 卫星计算函数 27
2.5.4 BDS 卫星计算函数 31
2.5.5 地球自转改正函数 33
2.6 误差改正 34
2.6.1 大气误差 34
2.6.2 粗差探测 36
2.7 单点定位与测速 38
2.7.1 单点定位 38
2.7.2 单点测速 42
2.8 获取实时数据流 44

2.9 主函数 45
三、 结果精度分析 45
3.1 结果展示 45
3.2 精度分析 46
3.3 优化方向 51
3.3.1 引入载波平滑伪距 (Hatch 滤波器) 51
3.3.2 实施加权最小二乘法 (WLS) 51
3.3.3 卡尔曼滤波器 (KF) 52
四、 心得与体会 52

一、程序设计概述

1.1 软件功能

软件是基于 PC 平台开发的一款导航定位程序，主要面向使用 NovAtel OEM719 双频、双系统（GPS+BDS）板卡的应用场景。

软件首先能够实时、连续地从 OEM719 板卡中读取原始观测数据，包括两个频点的伪距、载波相位、多普勒频移以及卫星广播星历与钟差参数等。软件在获取数据后，立即执行单点定位和测速（SPP 和 SPV）算法的全部计算流程，推算出接收机当前在 WGS-84 坐标系下的三维位置、三维速度、接收机钟差以及相应的定位精度指标，并将这些结果以与输入采样率相同的频率实时输出。为了便于后续分析与展示，程序一方面把计算结果写入本地文件，另一方面同步在屏幕上滚动显示关键信息。

整体开发过程分为两个阶段实现：第一阶段先在 PC 环境中完成数据文件驱动的离线解算，重点验证和优化导航定位算法；第二阶段再与硬件对接，实现通过串口/网络实时采集数据并完成在线解算，重点解决数据通信与同步问题。

图 1.1 GNSS 板卡 司南 K708（左） 芯星通 UB480（右）

1.2 软件接口

本软件接口主要可分为硬件对接接口、数据输入接口和数据输出接口。

表 1.1 软件接口分类

接口类型	核心作用	数据交互方式
硬件对接接口	与 NovAtel OEM719 板卡建立连接，实时、连续获取双频原始观测数据	单向接收（从板卡读取）
数据输入接口	离线阶段通过文件交互替代硬件接口，为算法验证提供数据支撑	单向读取（从本地文件读取）
数据输出接口	实现计算结果的“存储”与“展示”，衔接数据计算与结果应用	双向输出（文件写入 + 屏幕显示）

1.2.1 硬件对接接口

硬件通信接口是本导航定位程序与 NovAtel OEM719 板卡建立数据连接的核心通道，其核心作用是实现原始观测数据的实时、连续获取。接口目前主要承担单向数据接收任务 —— 即从板卡读取双频（GPS+BDS）相关的原始数据，是后续定位测速算法计算的基础。接口的设计重点满足 “实时性” 和 “完整性” 要求：一方面要与板卡采样率同步，连续接收数据不中断，避免因延迟或丢包影响定位时效性；另一方面要完整解析板卡输出的全部数据字段，确保伪距、星历等关键参数不缺失，避免直接导致后续定位算法计算失效。

程序中接口采用网络通信（TCP），所使用的服务的 IP 地址为 47.114.134.129；端口号为 7190。

1.2.2 数据输入接口

数据输入接口在软件开发的第一阶段（离线解算）发挥核心作用，本质是通过文件交互替代硬件接口，为算法验证提供数据支撑。其交互逻辑为单向读取 —— 即从本地存储的文件中获取原始数据，这些文件需模拟 NovAtel OEM719 板卡的真实输出格式，包含双频、双系统的伪距、载波相位、星历等完整数据字

段。设计该接口的核心目的是在不依赖硬件的情况下，验证单点定位和测速（SPP 和 SPV）算法的正确性：通过读取预先存储的 “模拟板卡数据文件”，软件可复现真实场景下的数据输入流程，进而检验算法在不同数据条件下的计算结果（如位置、速度精度），为后续优化算法提供依据。接口的关键要求是 “格式兼容性”—— 文件中的数据结构、字段定义必须与 OEM719 板卡的真实输出一致，否则会导致离线解算结果与实际硬件场景脱节，失去算法验证的意义。

1.2.3 数据输出接口

数据输出接口实现结果的 “存储” 与 “展示” ，从功能上分为文件写入接口和屏幕显示接口两类：文件写入接口负责将计算结果以固定格式写入本地文件，输出内容包括 WGS-84 坐标系下的三维位置、三维速度、接收机钟差及定位精度指标等，且写入频率与输入采样率保持一致，这一设计既保证了数据的实时性，又为后续分析（如通过专业工具复盘定位效果）提供了完整的原始数据支撑。屏幕显示接口则聚焦于用户实时监控需求，通过滚动显示的方式在界面上呈现关键信息（如当前位置、定位精度），同样以与输入采样率相同的频率更新，确保用户能直观掌握定位状态。两类接口共同满足了 “结果留存” 和 “即时反馈” 的需求，是软件从 “数据计算” 到 “结果应用” 的关键衔接。

1.3 编程语言

软件所使用的编程环境 Microsoft Visual Studio 2022 与 MATLAB R2024b；编程语言为 C++与 MATLAB。主要利用 C++来编写程序的主体部分，MATLAB 则是根据解算的结果进行图表的绘制。

1.4 完成情况

打开 SPP&SPV.sln 文件，点击运行按钮即选择：0 为文件模式，1 为实时模式。

选择 0：

用户选择“0”文件模式后，软件将自动读取文件夹中原有的

oem719-202504011900-1.bin 二进制文件，并将卫星位置与接收机位置的计算结果输出在 output_file.txt 文本文件中。

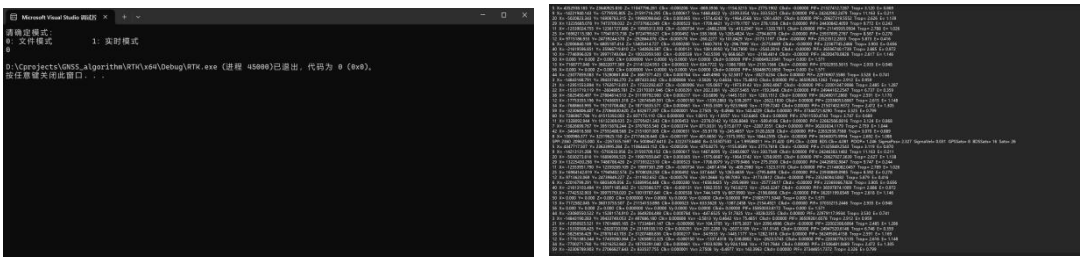

图 1.2 运行文件模式控制台界面（左） output_file.txt 输出文件（右）

选择 1：

用户选择“0”文件模式后，进行实时定位解算，并将接收机定位结果输出在 Windows 控制台上，并且软件会将定位精度与误差保存在 output_net.txt 文本文件中，用于后续使用 matlap 进行定位精度与误差分析。

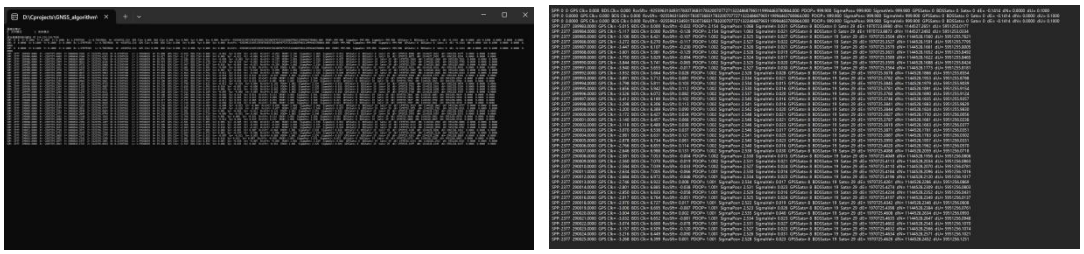

图 1.3 运行实时模式控制台界面（左） output_net.txt 输出文件（右）

二、算法与程序设计

2.1 总体设计

导航定位软件的流程设计如图 2.1。

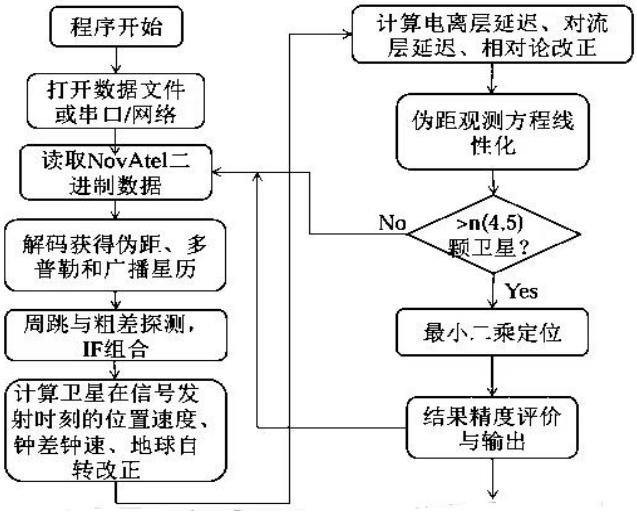

图 2.1 导航定位软件的流程设计

根据流程设计与功能逻辑，为提升程序的可维护性、可扩展性及开发效率，程序分为以下核心模块，各模块既独立承担特定功能，又通过预设接口实现数据交互与流程衔接：

表 2.1 导航定位软件的模块设计

文件类型	模块设计	文件名称
头文件（.h）	串口实时通信	Socket.h
	常量定义、结构体声明、函数声明	SPP.h
源文件（.cpp）	时间转换	TimeConverter.cpp
	坐标转换	CoordinateTransformation.cpp
	NovAtel OEM719 数据解码	Decode.cpp
	卫星位置计算	SatellitePosition.cpp
	误差改正	ErrorCorrection.cpp
	单点定位与测速	PVT.cpp
	主函数	main.cpp

2.2 时间转换

2.2.1 计时法与结构体定义

（1）通用计时法

表示方式：年、月、日、时、分、秒。

结构体设计：

```
struct COMMONTIME {
 short Year;
 unsigned short Month;
 unsigned short Day;
 unsigned short Hour;
 unsigned short Minute;
 double Second;

 COMMONTIME()
 {
  Year = 0;
  Month = 0;
  Day = 0;
  Hour = 0;
  Minute = 0;
  Second = 0.0;
 }
};
```

（2）简化儒略日

定义：儒略日是从-4712 年 1 月 1 日（公元前 4713 年 1 月 1 日）正午开始的天数；简化儒略日是从儒略日中减去 2,400,000.5 天来得到，给出的是从 1858 年 11 月 17 日子夜开始的天数。

结构体设计：

```
struct MJDTIME {
 int Days;//整数天
 double FracDay;//小数天

 MJDTIME()
 {
  Days = 0;
  FracDay = 0.0;
 }
```

```
};
```

（3）GPS 时

定义：以 1980 年 1 月 6 日子夜为起点，用周数和周内的秒数来表示。

结构体设计：

```
struct GPSTIME {
 unsigned short Week;
 double SecOfWeek;

 GPSTIME()
 {
  Week = 0;
  SecOfWeek = 0.0;
 }
};
```

2.2.2. 通用时转换为简化儒略日

2.2.2.1 算法原理

$$\mathrm{JD} = INT\left[365.25y\right] + INT\left[30.6001\left(m+1\right)\right] + D + \frac{UT}{24} + 1720981.5 \tag{2.1}$$

$$MJD = JD - 2400000.5 \tag{2.2}$$

其中，如果 $M \le 2$，则 $y = Y - 1$，$m = M + 12$；如果 $M > 2$，则 $y = Y$，$m = M$；。

式中，JD 为儒略日，MJD 为简化儒略日，Y、M、D、UT 分别为通用时的年、月、日和世界时。

2.2.2.2 函数设计

通用时转换为简化儒略日的函数设计如表 2.2：

表 2.2 通用时转换为简化儒略历的函数设计说明

函数	内容
名称	CommonTimeToMJDTime
声明	`void CommonTimeToMJDTime(const COMMONTIME* CT, MJDTIME* MJDT);`
功能	将通用时转化为简化儒略日

输入	COMMONTIME 结构体
输出	MJDTIME 结构体
注意事项	计算 MJD.FracDay 时，防止精度损失，直接用通用时中的时分秒计算小数天。

2.2.3 儒略日转换为通用时

2.2.3.1 算法原理

$$\mathrm{a} = INT\left(JD + 0.5\right) \tag{2.3}$$

$$b = a + 1537 \tag{2.4}$$

$$c = \mathrm{INT}\left[\left(b - 122.1\right)/365.25\right] \tag{2.5}$$

$$d = \mathrm{INT}\left[365.25c\right] \tag{2.6}$$

$$e = \mathrm{INT}\left[\left(b - d\right)/30.6001\right] \tag{2.7}$$

$$\mathrm{D} = b - d - INT\left(30.6001 \cdot e\right) + FRAC\left(JD + 0.5\right) \tag{2.8}$$

$$M = e - 1 - 12\mathrm{INT}\left[e/14\right] \tag{2.9}$$

$$Y = c - 4715 - \mathrm{INT}\left[\left(7 + M\right)/10\right] \tag{2.10}$$

$$N = \mathrm{mod}\left\{\mathrm{INT}\left[JD + 0.5\right], 7\right\} \tag{2.11}$$

其中，Y、M、D、N 分别为通用时的年、月、日和周。

2.2.3.2 函数设计

儒略日转换为通用时的函数设计如表 2.3：

表 2.3 儒略日转换为通用时的函数设计说明

函数	内容
名称	MJDTimeToCommonTime
声明	`void MJDTimeToCommonTime(const MJDTIME* MJDT, COMMONTIME* CT);`

功能	将儒略日转换为通用时
输入	MJDTIME 结构体
输出	COMMONTIME 结构体
注意事项	计算通用时中的时分秒，防止精度损失，直接用 MJD.FracDay 计算。

2.2.4 简化儒略日转换为 GPS 时

2.2.4.1 算法原理

$$GPSWEEK = \mathrm{INT}\left[\left(MJD - 44244\right)/7\right] \tag{2.12}$$

$$SECOFWEEK = \left(MJD - 44244 - GPSWEEK * 7\right) * 86400 \tag{2.13}$$

其中，GPSWEEK 为 GPS 周，SECOFWEEK 为 GPS 周秒。

2.2.4.2 函数设计

简化儒略日转换为 GPS 时的函数设计如表 2.4：

表 2.4 简化儒略日转换为 GPS 时的函数设计说明

函数	内容
名称	MJDTimeToGPSTime
声明	`void MJDTimeToGPSTime(const MJDTIME* MJDT, GPSTIME* GT);`
功能	将简化儒略日转换为 GPS 时
输入	MJDTIME 结构体
输出	GPSTIME 结构体

2.2.5 GPS 时转换为简化儒略日

2.2.5.1 算法原理

$$MJD = 44244 + GPSWEEK * 7 + SECOFWEEK / 86400 \tag{2.14}$$

2.2.5.2 函数设计

GPS 时转换为简化儒略日的函数设计如表 2.5：

表 2.5 GPS 时转换为简化儒略日的函数设计说明

函数	内容
名称	GPSTimeToMJDTime
声明	`void GPSTimeToMJDTime(const GPSTIME* GT, MJDTIME* MJDT);`
功能	将 GPS 时转换为简化儒略日
输入	GPSTIME 结构体
输出	MJDTIME 结构体

2.2.6 通用时与 GPS 时的相互转换

2.2.6.1 算法原理

通用时到 GPS 时的转换算法：由通用时转换到简化儒略日，再由简化儒略日转换到 GPS 时。

GPS 时到通用时的转换算法：由 GPS 时转换到儒略日，再由儒略日转换到通用时。

2.2.6.2 函数设计

通用时与 GPS 时相互转换的两个函数设计如表 2.6：

表 2.6 通用时与 GPS 时相互转换的函数设计说明

函数	内容 1	内容 2
名称	CommonTimeToGPSTime	GPSTimeToCommonTime
声明	`void CommonTimeToGPSTime(const COMMONTIME* CT, GPSTIME* GT);`	`void GPSTimeToCommonTime(const GPSTIME* GT, COMMONTIME* CT);`
功能	将通用时转换为 GPS 时	将 GPS 时转换为通用时
输入	COMMONTIME 结构体	GPSTIME 结构体

输出	GPSTIME 结构体	COMMONTIME 结构体

2.3 坐标转换

2.3.1 大地坐标转换到笛卡尔坐标

2.3.1.1 算法原理

$$X=\left(N+H\right)\cos B\cos L \tag{2.15}$$

$$Y=\left(N+H\right)\cos B\sin L \tag{2.16}$$

$$Z=\left[N\left(1-e^2\right)+H\right]\sin B=\left[N\cdot\frac{b^2}{a^2}+H\right]\sin B \tag{2.17}$$

其中，N 为卯酉圈的半径，$N=\frac{a}{\sqrt{1-e^2\sin^2 B}}$；$e$ 为第一偏心率，$e^2=\frac{a^2-b^2}{a^2}=2f-f^2$；X、Y、Z 为大地坐标，单位为米；B，L，H 为笛卡尔坐标，B，L 的单位为弧度，H 的单位为米；a 为地球半径，f 为扁率。

2.3.1.2 函数设计

大地坐标系转换到笛卡尔坐标系的函数设计如表 2.7：

表 2.7 大地坐标系转换到笛卡尔坐标系的函数设计说明

函数	内容
名称	BLHToXYZ
声明	`void BLHToXYZ(const double BLH[3], double XYZ[3], const double R, const double F);`
功能	大地坐标系转换到笛卡尔坐标系
输入	大地坐标 BLH[3]，半径 R，扁率 F
输出	笛卡尔坐标 XYZ[3]

2.3.2 笛卡尔坐标转换到大地坐标

2.3.2.1 算法原理

$$\mathrm{B}=\arctan\left(\frac{Z+\Delta Z}{\sqrt{X^2+Y^2}}\right) \tag{2.18}$$

$$L=\arctan\left(\frac{Y}{X}\right) \tag{2.19}$$

$$H=\sqrt{X^2+Y^2+\left(Z+\Delta Z\right)^2}-N \tag{2.20}$$

其中，$\Delta Z=Ne^2\sin B$ 。

2.3.2.2 函数设计

笛卡尔坐标系转换到大地坐标系的函数设计如表 2.8：

表 2.8 笛卡尔坐标系转换到大地坐标系的函数设计说明

函数	内容
名称	XYZToBLH
声明	`void XYZToBLH(const double xyz[3], double blh[3], const double R, const double F);`
功能	笛卡尔坐标系转换到大地坐标系
输入	笛卡尔坐标 XYZ[3]，半径 R，扁率 F
输出	大地坐标 BLH[3]

2.3.3 卫星高度角和方位角计算

2.3.3.1 算法原理

$$\begin{bmatrix}\mathrm{dE}\\ dN\\ dH\end{bmatrix}=R\begin{bmatrix}x^{\mathrm{s}}-x_0\\ y^{\mathrm{s}}-y_0\\ z^{\mathrm{s}}-z_0\end{bmatrix}=\begin{bmatrix}-\sin L & \cos L & 0\\ -\sin B\cos L & -\sin B\sin L & \cos B\\ \cos B\cos L & \cos B\sin L & \sin B\end{bmatrix}\begin{bmatrix}x^{\mathrm{s}}-x_0\\ y^{\mathrm{s}}-y_0\\ z^{\mathrm{s}}-z_0\end{bmatrix} \tag{2.21}$$

$$\text{Elev} = \text{atan}\left(\frac{dH}{\sqrt{dN^2 + dE^2}}\right) \tag{2.22}$$

$$Azim = a\tan 2\left(dE, dN\right) \tag{2.23}$$

其中，x_0，y_0，z_0 为测站坐标；B，L，H 为测站的大地坐标；x^s，y^s，z^s 为卫星坐标；$Elev$ 为卫星高度角，$Azim$ 为卫星方位角。

2.3.3.2 函数设计

卫星高度角和方位角计算的函数设计如表 2.9：

表 2.9 卫星高度角和方位角计算的函数设计说明

函数	内容
名称	CompSatElAz
声明	`void CompSatElAz(const double Xr[], const double Xs[], const double R, const double F, double* Elev, double* Azim);`
功能	卫星高度角和方位角计算
输入	测站坐标，卫星坐标，半径 R，扁率 F
输出	高度角 Elev，方位角 Azim

2.3.4 定位误差计算

2.3.4.1 算法原理

$$\begin{bmatrix} \text{dE} \\ dN \\ dH \end{bmatrix} = R\begin{bmatrix} x - x_0 \\ y - y_0 \\ z - z_0 \end{bmatrix} = \begin{bmatrix} -\sin L & \cos L & 0 \\ -\sin B\cos L & -\sin B\sin L & \cos B \\ \cos B\cos L & \cos B\sin L & \sin B \end{bmatrix}\begin{bmatrix} x - x_0 \\ y - y_0 \\ z - z_0 \end{bmatrix} \tag{2.24}$$

其中，x_0，y_0，z_0 为测站的精确坐标；x，y，z 为测站的计算坐标

2.3.4.2 函数设计

定位误差计算的函数设计如表 2.9：

表 2.9 定位误差计算的函数设计说明

函数	内容
名称	Comp_dEnu
声明	`void Comp_dEnu(const double X0[], const double Xr[], const double R, const double F, double dNeu[]);`
功能	定位误差计算
输入	测站精确坐标，测站计算坐标，半径 R，扁率 F
输出	地平坐标系下的定位误差（dE, dN, dH）

2.4 NovAtel OEM719 数据解码

该模块主要用于读取 NovAtel OEM719 数据格式下的观测数据、星历数据和接收机定位结果。模块的流程设计如图 2.2。

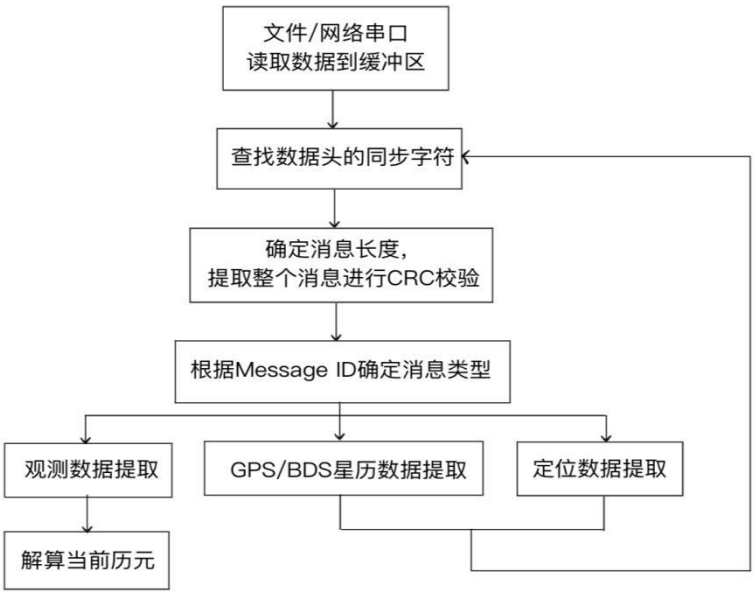

图 2.2 数据解码模块流程设计

根据模块设计定义需要使用的结构体，包括观测数据结构体、卫星星历结构体和定位结果结构体：

（1）观测数据结构体

```
//定义每颗卫星的观测数据
struct SATOBSDATA {
 short Prn;
 GNSSSys System;
 double P[2], L[2], D[2];
 double cn[2], LockTime[2];  //载噪比，锁定时长
 unsigned char half[2];      //相位锁定
 bool Valid;                 //标记值

 SATOBSDATA()
 {
  Prn = 0;
  System = UNKS;
  for(int i=0;i<2;i++)
  {
   P[i] = L[i] = D[i] = 0.0;
  }
  Valid = false;
 }
};
```

```
//定义每个历元的观测数据
struct EPOCHOBSDATA {
 GPSTIME Time;      //从头文件中获取
 short SatNum;
 SATOBSDATA SatObs[MAXCHANNUM];

 EPOCHOBSDATA()
 {
  SatNum = 0;
 }
};
```

（2）卫星星历结构体

```
struct GPSEPHREC {
 short Prn;
 GNSSSys Sys;
 GPSTIME TOC, TOE;          //卫星钟参考时刻，星历（轨道）参考时刻（s）
 double ClkBias, ClkDrift, ClkDriftRate;       //卫星钟的偏差(s)、漂移、漂移速度
 double IODE, IODC;                            //星历发布时间，钟的数据龄期
```

```
    double SqrtA, M0, e, OMEGA, i0, omega;          //卫星轨道根数
    double Crs, Cuc, Cus, Cic, Cis, Crc;            //短周期摄动项系数(m,rad,rad,rad,rad,m)
    double DeltaN, OMEGADot, iDot;                  //平均角速度改正数（rad/s),长期摄动项参数（rad/s)，iDot（rad/s)
    int SVHealth;                 //BDS:0 is good
    double TGD1, TGD2;          //群延迟（s,GPS 一个，BDS 两个）

    GPSEPHREC() {
     Prn = 0;
     SVHealth = 1;
     Sys = UNKS;
     ClkBias = ClkDrift = ClkDriftRate = IODE = IODC = TGD1 = TGD2 = 0.0;
     SqrtA = e = M0 = OMEGA = i0 = omega = OMEGADot = iDot = DeltaN = 0.0;
     Crs = Cuc = Cus = Cic = Cis = Crc = 0.0;
    }
  };
```

（3）定位结构体

```
  struct POSRES {
    GPSTIME Time;
    double Pos[3], Vel[3];
    double PDOP, SigmaPos, SigmaVel;          //精度指标
    int SatNum;

    POSRES()
    {
     for (int i = 0; i < 3; i++)
     {
      Pos[i] = Vel[i] = 0.0;
     }
     PDOP = SigmaPos = SigmaVel = 999.9;
     GPS_SatNum = 0;
    }
  };
```

2.4.1 解码主函数

2.4.1.1 二进制数据格式

二进制数据的基本结构是消息头、消息体、CRC 校验码。

消息头的起始为 3 个同步字节 AA 44 12，根据这个同步字节来判断是否为一条新消息。程序中需要使用的消息头中的信息如表 2.10 所示：

表 2.10 二进制消息头格式

域#	域名	域的类型	描述	二进制字节数	二进制偏移量
1	Sync	Char	Hexadecimal 0xAA	1	0
2	Sync	Char	Hexadecimal 0x44	1	1
3	Sync	Char	Hexadecimal 0x12	1	2
4	Header Lgth	Uchar	消息头的长度	1	3
5	Message ID	Ushort	消息 ID，根据消息 ID 判断该条消息的类型	2	4
8	Message Length	Ushort	消息体的长度	2	8
12	Week	Ushort	GPS 周	2	14
13	ms	GPSec	GPS 周内秒（ms）	4	16

消息体有多种类型，本软件设计需要用到的包括观测值、GPS 星历、BDS 星历和定位结果。

表 2.11 消息体类型以及 Message ID 表格

消息体类型	Message ID
观测值	43
GPS 星历	7
BDS 星历	1696
定位结果	42

CRC 校验码，全称 Cyclic Redundancy Check，意味循环冗余校验，目的是验证数据在通信过程中是否存在错误，具有很高的可靠性。CRC 校验的原理如下：

（1）被校验的原数据转换成二进制序列，假设一共有 K 位；

（2）以一定的规则产生一个新的二进制序列，假设共 R 位，即 CRC 校验码；

（3）把新的二进制序列附加在原数据二进制序列后，一共 K+R 位，发送出

去；

（4）接收端接收到数据后，把原数据的 K 位二进制序列按照相同的规则产生一个 R 位二进制序列与附加的 R 位二进制序列进行比较，相同表示传递的数据没问题，不相同表示传递的数据出现错误与误差。

2.4.1.2 解码主函数设计

根据二进制数据格式，解码主函数的设计思路如下：

（1）设置循环变量 i=0，开始查找 AA 44 12 同步字符；

（2）找到同步字符后，获取消息头长度的字符 28 字节，若字节数量不足即 i+28>len，跳出循环（break）至第 6 步；

（3）从消息头中解码消息长度 MsgLen 和消息类型 MsgID，获得整条消息 buff[i, i+28+MsgLen+4]，若字节数量不足即 i+28+MsgLen+4>len，跳出循环（break）至第 6 步；

（4）CRC 检验，若不通过，跳过同步字符 3 个字节，即 i=i+3，返回到第 1 步；

（5）根据消息 ID，调用对应的解码函数，若解码得到观测值，跳出循环至第 6 步；否则，跳过整条消息，即 i= i+28+MsgLen+4，返回第 1 步；

（6）循环结束后，将不足一条消息的剩余字节，拷贝至 buff 缓冲区的开始处，将 len 设置为剩余字节数量，并返回给主函数。

程序中，解码主函数、CRC 校验函数、字节提取函数设计分别如表 2.12、2.13、2.14。

表 2.12 解码主函数设计说明

函数	内容
名称	DecodeNovOem7Dat
声明	`int DecodeNovOem7Dat(unsigned char Buff[], int& Len, EPOCHOBSDATA* obs, GPSEPHREC geph[], GPSEPHREC beph[], POSRES* pos);`
功能	板卡解码主函数
输入	缓冲区 Buff[]，缓冲区长度 Len
输出	观测值，GPS 星历，北斗星历，定位结果

表 2.13 CRC 校验函数设计说明

函数	内容
名称	crc32
声明	`unsigned int crc32(const unsigned char* buff, int len);`
功能	进行 CRC 校验
输入	缓冲区的起始地址，缓冲区长度
输出	CRC 校验码

表 2.14 字节提取函数总列表

名称	功能
S2	提取 2 字节有符号整数
US2	提取 2 字节无符号整数
I4	提取 4 字节有符号整数
UI4	提取 4 字节无符号整数
F4	提取 4 字节 float
D8	提取 8 字节 double

2.4.2 观测数据解码函数

2.4.2.1 观测数据格式

观测值的 Message ID 为 43，里面主要存储当前历元观测到的每颗卫星的观测值信息，观测数据格式如图 2.3 所示：

Field	Field type	Description	Format	Binary Bytes	Binary Offset
1	RANGE header	Log header. See *Messages* on page 30 for more information.		H	0
2	# obs	Number of observations with information to follow [1]	Ulong	4	H
3	PRN/slot	Satellite PRN number of range measurement Refer to *PRN Numbers* on page 49	Ushort	2	H+4
4	glofreq	(GLONASS Frequency + 7) (see GLONASS Slot and Frequency Numbers section of this manual)	Ushort	2	H+6
5	psr	Pseudorange measurement (m)	Double	8	H+8
6	psr σ	Pseudorange measurement standard deviation (m)	Float	4	H+16
7	adr	Carrier phase, in cycles (accumulated Doppler range)	Double	8	H+20
8	adr σ	Estimated carrier phase standard deviation (cycles)	Float	4	H+28
9	dopp	Instantaneous carrier Doppler frequency (Hz)	Float	4	H+32
10	C/No	Carrier to noise density ratio C/No = 10[log10(S/N_0)] (dB-Hz)	Float	4	H+36
11	locktime	Number of seconds of continuous tracking (no cycle slipping)	Float	4	H+40
12	ch-tr-status	Tracking status (see *Table 149: Channel Tracking Status* on the next page and the example in *Figure 15: Channel Tracking Example* on the next page)	Ulong	4	H+44
13...	Next PRN offset = H + 4 + (#obs x 44)				
variable	xxxx	32-bit CRC (ASCII and Binary only)	Ulong	4	H+4+ (#obs x 44)
variable	[CR][LF]	Sentence terminator (ASCII only)	-	-	-

	N7				N6				N5				N4				N3				N2				N1				N0			
0x	0				8				1				0				9				C				0				4			
Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Binary	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	1	0	0	1	1	1	0	0	0	0	0	0	0	1	0	0
Data	Channel Assignment	Lock Flag	Digital filtering on signal	Phase Measurement	Primary L1	Reserved	Signal Type					Grouping	Reserved	Satellite System			Correlator Spacing			Code Locked Flag	Parity Flag	Phase Lock Flag	Channel Number						Tracking State			
Value	Automatic	Lock Out	No Digital Filter	Half Cycle	Primary		L1 C/A					Grouped		GPS			PAC			Locked	Known	Locked	Channel 0						L1 Phase Lock Loop			

图 2.3 观测数据格式

2.4.2.2 观测数据解码函数设计

根据观测数据格式，观测数据解码函数的设计思路如下，注意清空之前的数

据：

（1）从消息头中解码得到观测时刻，该时刻为接收机钟表面时，用 GPSTIME 结构体表示；

（2）解码得到观测值数量，为所有卫星所有信号观测值的总数；

（3）对所有信号观测值进行循环解码；

（4）统计有效的卫星数量。

程序中，观测数据解码函数设计如表 2.15：

表 2.15 观测数据解码函数设计说明

函数	内容
名称	decode_rangeb_oem7
声明	`int decode_rangeb_oem7(unsigned char* Buff, EPOCHOBSDATA* obs);`
功能	观测数据解码
输入	缓冲区 Buff[]
输出	观测值，解码成功返回值为 1

2.4.3 GPS 星历解码函数

2.4.3.1 GPS 星历格式

GPSEHPEM 消息的 Message ID 为 7，里面主要存储每颗 GPS 卫星的广播星历，GPS 广播星历每两小时更新一次，GPS 星历格式如图 2.4 所示：

Field	Field type	Description	Format	Binary bytes	Binary offset
1	Log header	GPSEPHEM header For information about log headers, see ASCII, Abbreviated ASCII or Binary.		H	0
2	PRN	Satellite PRN number	Ulong	4	H
3	tow	Time stamp of subframe 1 (s)	Double	8	H+4
4	health	Health status • bits 0-5: 6-bit health code as defined in IS-GPS-200. • bit 6: Alert flag	Ulong	4	H+12
5	IODE1	Issue of ephemeris data 1	Ulong	4	H+16
6	IODE2	Issue of ephemeris data 2	Ulong	4	H+20
7	week	toe week number computed from Z count week (GPS reference week number)	Ulong	4	H+24
8	z week	Z count week number. This is the week number from subframe 1 of the ephemeris. The 'toe week' (field #7) is derived from this to account for rollover	Ulong	4	H+28
9	toe	Reference time for ephemeris (s)	Double	8	H+32
10	A	Semi-major axis (m)	Double	8	H+40
11	ΔN	Mean motion difference (radians/s)	Double	8	H+48
12	M_0	Mean anomaly of reference time (radians)	Double	8	H+56
13	ecc	Eccentricity, dimensionless	Double	8	H+64
14	ω	Argument of perigee (radians)	Double	8	H+72
15	cuc	Amplitude of cosine harmonic correction term to the argument of latitude (radians)	Double	8	H+80
16	cus	Amplitude of sine harmonic correction term to the argument of latitude (radians)	Double	8	H+88
17	crc	Amplitude of cosine harmonic correction term to the orbit radius (m)	Double	8	H+96
18	crs	Amplitude of sine harmonic correction term to the orbit radius (m)	Double	8	H+104
19	cic	Amplitude of cosine harmonic correction term to the angle of inclination (radians)	Double	8	H+112
20	cis	Amplitude of sine harmonic correction term to the angle of inclination (radians)	Double	8	H+120
21	i_0	Inclination angle at reference time (radians)	Double	8	H+128
22	$\dot{i}$	Rate of inclination angle (radians/s)	Double	8	H+136
23	Ω_0	Longitude of ascending node of orbit plane (radians)	Double	8	H+144
24	$\dot{\omega}$	Rate of right ascension (radians/s)	Double	8	H+152
25	iodc	Issue of data clock	Ulong	4	H+160
26	toc	SV clock correction term (s)	Double	8	H+164
27	tgd	Estimated group delay difference (s)	Double	8	H+172
28	a_0	Clock aging parameter (s)	Double	8	H+180
29	a_1	Clock aging parameter (s/s)	Double	8	H+188
30	a_2	Clock aging parameter (s/s/s)	Double	8	H+196
31	AS	Anti-spoofing on: 0 = FALSE 1 = TRUE	Bool	4	H+204
32	N	Corrected mean motion (radians/s) This field is computed by the receiver.	Double	8	H+208
33	URA	User Range Accuracy variance $(m)^2$ The ICD specifies that the URA index transmitted in the ephemerides can be converted to a nominal standard deviation value using an algorithm listed there. We publish the square of the nominal value (variance). The correspondence between the original URA index and the value output is shown in Table: URA variance.	Double	8	H+216
34	xxxx	32-bit CRC (ASCII and Binary only)	Ulong	4	H+224
35	[CR][LF]	Sentence terminator (ASCII only)	-	-	-

图 2.4 GPS 星历格式

2.4.3.2 GPS 星历解码函数设计

根据 GPS 星历格式，GPS 星历解码函数的设计思路如下

(1)解码得到卫星号 prn，判断 prn-1 是否超出 Eph 数组范围。如果越界函数返回;

(2)根据 OEM 星历格式，解码星历数据，注意各数据的单位。

程序中，GPS 星历解码函数设计如表 2.16：

表 2.16 GPS 星历解码函数设计说明

函数	内容
名称	decode_gpsephem
声明	`int decode_gpsephem(unsigned char* buff, GPSEPHREC* geph);`
功能	GPS 星历解码
输入	缓冲区 Buff[]
输出	GPS 星历，解码成功返回值为 2，否则返回值为 0

2.4.4 BDS 星历解码函数

2.4.4.1 BDS 星历格式

BDSEHPEMRIS 消息的 Message ID 为 1696，里面主要存储每颗 BDS 卫星的广播星历，BDS 广播星历每小时更新一次，程序中需要使用的 BDS 星历格式如图 2.5 所示：

Field	Field type	Description	Format	Binary bytes	Binary offset
1	Log header	BDSEPHEMERIS header For information about log headers, see ASCII, Abbreviated ASCII or Binary.		H	0
2	satellite ID	ID/ranging code	Ulong	4	H
3	Week	Week number in BeiDou time (BDT) since the BeiDou system time start epoch (January 1, 2006)	Ulong	4	H+4
4	URA	User range accuracy (m). This is the evaluated URAI/URA lookup-table value.	Double	8	H+8
5	health 1	Autonomous satellite health flag. 0 means broadcasting satellite is good and 1 means not.	Ulong	4	H+16
6	tgd1	Equipment group delay differential for the B1 signal (s)	Double	8	H+20
7	tgd2	Equipment group delay differential for the B2 signal (s)	Double	8	H+28
8	AODC	Age of data, clock	Ulong	4	H+36
9	toc	Reference time of clock parameters (s)	Ulong	4	H+40
10	a_0	Constant term of clock correction polynomial (s)	Double	8	H+44
11	a_1	Linear term of clock correction polynomial (s/s)	Double	8	H+52
12	a_2	Quadratic term of clock correction polynomial (s/s^2)	Double	8	H+60
13	AODE	Age of data, ephemeris	Ulong	4	H+68
14	toe	Reference time of ephemeris parameters (s)	Ulong	4	H+72
15	RootA	Square root of semi-major axis (sqrt(m))	Double	8	H+76
16	ecc	Eccentricity (dimensionless)	Double	8	H+84
17	ω	Argument of perigee (radians)	Double	8	H+92
18	ΔN	Mean motion difference from computed value (radians/s)	Double	8	H+100
19	M_0	Mean anomaly at reference time (radians)	Double	8	H+108
20	Ω_0	Longitude of ascending node of orbital of plane computed according to reference time (radians)	Double	8	H+116
21	$\dot{\Omega}$	Rate of right ascension (radians/s)	Double	8	H+124
22	i_0	Inclination angle at reference time (radians)	Double	8	H+132
23	IDOT	Rate of inclination angle (radians/second)	Double	8	H+140
24	c_{uc}	Amplitude of cosine harmonic correction term to the argument of latitude (radians)	Double	8	H+148
25	c_{us}	Amplitude of sine harmonic correction term to the argument of latitude (radians)	Double	8	H+156
26	c_{rc}	Amplitude of cosine harmonic correction term to the orbit radius (m)	Double	8	H+164
27	c_{rs}	Amplitude of sine harmonic correction term to the orbit radius (m)	Double	8	H+172
28	c_{ic}	Amplitude of cosine harmonic correction term to the angle of inclination (radians)	Double	8	H+180
29	c_{is}	Amplitude of sine harmonic correction term to the angle of inclination (radians)	Double	8	H+188
30	xxxx	32-bit CRC (ASCII and Binary only)	Ulong	4	H+196
31	[CR][LF]	Sentence terminator (ASCII only)	-	-	-

图 2.5 BDS 星历格式

2.4.4.2 BDS 星历解码函数设计

在 GPS 星历解码函数的设计思路基础上，BDS 星历解码还需要注意 BDS 星历的参考时间 TOC 和 TOE 均定义为 GPSTIME 结构体，时间系统为北斗时。

程序中，BDS 星历解码函数设计如表 2.17：

表 2.17 BDS 星历解码函数设计说明

函数	内容
名称	decode_bdsephem
声明	`int decode_bdsephem(unsigned char* buff, GPSEPHREC* beph);`
功能	BDS 星历解码
输入	缓冲区 Buff[]
输出	BDS 星历，解码成功返回值为 3，否则返回值为 0

2.4.5 定位结果解码函数

2.4.5.1 定位结果格式

BESTPOS 消息的 Message ID 为 42，里面主要存储最佳的定位结果，定位结果结构格式 2.6 所示：

Field	Field type	Description	Format	Binary bytes	Binary offset
1	Log header	BESTPOS header For information about log headers, see ASCII, Abbreviated ASCII or Binary.		H	0
2	sol stat	Solution status, see Table: Solution status	Enum	4	H
3	pos type	Position type, see Table: Position or velocity type	Enum	4	H+4
4	lat	Latitude (degrees)	Double	8	H+8
5	lon	Longitude (degrees)	Double	8	H+16
6	hgt	Height above mean sea level (metres)	Double	8	H+24
7	undulation	Undulation - the relationship between the geoid and the ellipsoid (m) of the chosen datum When using a datum other than WGS84, the undulation value also includes the vertical shift due to differences between the datum in use and WGS84.	Float	4	H+32
8	datum id#	Datum ID number 61 = WGS84 63 = USER	Enum	4	H+36
9	lat σ	Latitude standard deviation (m)	Float	4	H+40
10	lon σ	Longitude standard deviation (m)	Float	4	H+44
11	hgt σ	Height standard deviation (m)	Float	4	H+48
12	stn id	Base station ID	Char[4]	4	H+52
13	diff_age	Differential age in seconds	Float	4	H+56
14	sol_age	Solution age in seconds	Float	4	H+60
15	#SVs	Number of satellites tracked	Uchar	1	H+64
16	#solnSVs	Number of satellites used in solution	Uchar	1	H+65
17	#solnL1SVs	Number of satellites with L1/E1/B1 signals used in solution	Uchar	1	H+66
18	#solnMultiSVs	Number of satellites with multi-frequency signals used in solution	Uchar	1	H+67
19	Reserved		Hex	1	H+68
20	ext sol stat	Extended solution status (see Table: Extended solution status)	Hex	1	H+69
21	Galileo and BeiDou sig mask	Galileo and BeiDou signals used mask (see Table: Galileo and BeiDou signal-used mask)	Hex	1	H+70
22	GPS and GLONASS sig mask	GPS and GLONASS signals used mask (see Table: GPS and GLONASS signal-used mask)	Hex	1	H+71
23	xxxx	32-bit CRC (ASCII and Binary only)	Hex	4	H+72
24	[CR][LF]	Sentence terminator (ASCII only)	-	-	-

图 2.6 定位结果格式

2.4.5.2 定位结果函数设计

程序中，定位结果解码函数设计如表 2.18：

表 2.18 定位结果解码函数设计说明

函数	内容
名称	decode_psrpos
声明	`int decode_psrpos(unsigned char* buff, POSRES* pos);`
功能	定位结果解码
输入	缓冲区 Buff[]
输出	定位结果，解码成功返回值为 4，否则返回值为 0

2.5 卫星位置计算

该模块主要通过对多个子程序，对每颗卫星观测值进行循环，计算所有观测卫星的 PVT 等信息。卫星位置计算模块的流程设计如图 2.7。

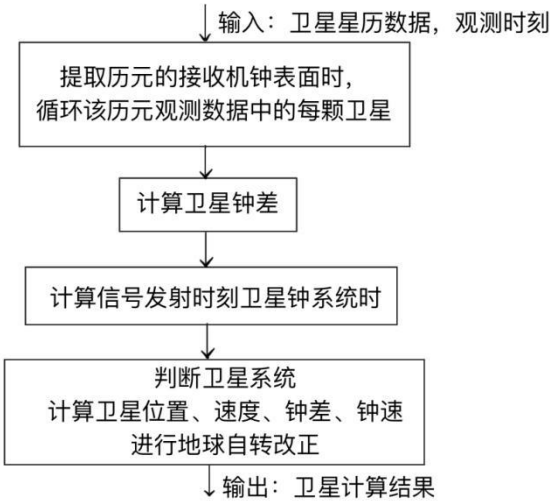

图 2.7 卫星位置计算模块流程设计

根据模块设计思路，除了需要使用卫星星历结构体外，还需要定义卫星计算结果结构体：

```
struct SATMIDRES
{
 double SatPos[3], SatVel[3];
 double SatClkOft, SatClkSft;
 double Elevation, Azimuth;
 double TropCorr;
 double Tgd1, Tgd2;
 bool Valid;  //false=没有星历或星历过期,true=计算成功
```

```
SATMIDRES()
{
 SatPos[0] = SatPos[1] = SatPos[2] = 0.0;
 SatVel[0] = SatVel[1] = SatVel[2] = 0.0;
 Elevation = PAI / 2.0;
 SatClkOft = SatClkSft = 0.0;
 Tgd1 = Tgd2 = TropCorr = 0.0;
 Valid = false;
}
};
```

2.5.1 卫星位置计算主函数

在卫星位置计算主函数中，通过对卫星钟差计算函数、GPS 卫星计算函数、BDS 卫星计算函数和地球自转改正函数的调用，循环计算出每颗卫星的位置、速度、钟差和钟速。程序中，函数设计如表 2.19：

表 2.19 卫星位置计算主函数设计说明

函数	内容
名称	SatellitePosition
声明	`void SatellitePosition(GPSEPHREC geph[], GPSEPHREC beph[], EPOCHOBSDATA* obs, double UserPos[3]);`
功能	计算某历元观测中所有卫星的位置、速度、钟差和钟速
输入	某历元的观测值和卫星星历
输出	obs 中的卫星位置、速度、钟差、钟速的计算结果

2.5.2 卫星钟差计算函数

2.5.2.1 算法原理

（1）进行有效性判断 $\mu = 3.986005 \times 10^{14} m^3 / s^2$

$$\text{Valid} = \begin{cases} \text{T}, & |\Delta t| \le T_{\max} \text{ 且 } SVHealth = 0 \\ \text{F}, & \text{否则} \end{cases} \tag{2.25}$$

其中对于 GPS 系统，星历有效期 $T_{\max}$ 为 2 小时约等于 7500s；对于 BDS 系

统，星历有效期为一小时约等于 3900s。

（2）计算卫星钟差

$$\Delta t=\left(t_{obs}^{week}-t_{TOC}^{week}\right)\cdot 604800+\left(t_{obs}^{\sec}-t_{TOC}^{\sec}\right) \tag{2.26}$$

$$\Delta t_{SV}=ClkBias+ClkDrift\cdot\Delta t+ClkDriftRate\cdot\Delta t^{2} \tag{2.27}$$

注意 BDS 星历的参考时间为 BDT，$t_{BDT}=t_{GPST}-1356\text{周}-14\text{秒}$。

2.5.2.2 函数设计

程序中，卫星钟差计算函数设计如表 2.20：

表 2.20 卫星钟差计算函数设计说明

函数	内容
名称	CompSatClkOff
声明	bool CompSatClkOff(const int Prn, const GNSSSys Sys, const GPSTIME* t, GPSEPHREC* GPSEph, GPSEPHREC* BDSEph, SATMIDRES* Mid);
功能	计算卫星钟差
输入	Prn 号，Sys 卫星导航系统， 卫星信号发射时刻，卫星星历数据
输出	不加相对论改正的卫星钟差

2.5.3 GPS 卫星计算函数

2.5.3.1 算法原理

（1）GPS 卫星位置

GPS 系统下的地球重力常数，地球自转角速度 $\dot{\Omega}_e=7.2921151467\times10^{-5}rad/s$。

计算轨道长半轴：

$$A=\left(\sqrt{A}\right)^{2} \tag{2.28}$$

计算平均运动角速度：

$$n_0 = \sqrt{\frac{\mu}{A^3}} \tag{2.29}$$

计算相对于星历参考历元的时间：

$$t_k = t - t_{oe} \tag{2.30}$$

对平均运动角速度进行改正：

$$n = n_0 + \Delta n \tag{2.31}$$

计算平近点角：

$$M_k = M_0 + nt_k \tag{2.32}$$

迭代计算偏近点角：

$$M_k = E_k - e\sin E_k \tag{2.33}$$

计算真近点角：

$$v_k = \arctan\left(\frac{\sin v_k}{\cos v_k}\right) = \arctan\left(\frac{\sqrt{1-e^2}\sin E_k / \left(1 - e\cos E_k\right)}{\left(\cos E_k - e\right)\left(1 - \mathrm{e}\cos E_k\right)}\right) \tag{2.34}$$

计算升交角距：

$$\Phi_k = v_k + \omega \tag{2.35}$$

计算升交角距的改正数：

$$\delta u_k = C_{us}\sin 2\Phi_k + C_{uc}\cos 2\Phi_k \tag{2.36}$$

计算向径的改正数：

$$\delta r_k = C_{rs}\sin 2\Phi_k + C_{rc}\cos 2\Phi_k \tag{2.37}$$

计算轨道倾角的改正数：

$$\delta i_k = C_{is}\sin 2\Phi_k + C_{ic}\cos 2\Phi_k \tag{2.38}$$

计算经过改正的升交角距：

$$u_k = \Phi_k + \delta u_k \tag{2.39}$$

计算经过改正的向径：

$$r_k = A\left(1 - e\cdot\cos E_k\right) + \delta r_k \tag{2.40}$$

计算经过改正的轨道倾角：

$$i_k = i_0 + \delta i_k + \left(IDOT\right)\cdot t_k \tag{2.41}$$

计算卫星在轨道平面上的位置：

$$\begin{cases} x_k^{'} = r_k \cos u_k \\ y_k^{'} = r_k \sin u_k \end{cases} \tag{2.42}$$

计算改正后的升交点经度：

$$\Omega_k = \Omega_0 + \left(\dot{\Omega} - \dot{\Omega}_e\right)\cdot t_k - \dot{\Omega}_e \cdot t_{oe} \tag{2.43}$$

计算在地固坐标系下的位置：

$$\begin{cases} x_k = x_k^{'} \cos\Omega_k - y_k^{'} \cos i_k \sin\Omega_k \\ y_k = x_k^{'} \sin\Omega_k + y_k^{'} \cos i_k \cos\Omega_k \\ z_k = y_k^{'} \sin i_k \end{cases} \tag{2.44}$$

（2）GPS 卫星速度

计算偏近点角的变化率：

$$\dot{E}_k = \frac{n}{1 - e\cos E_k} \tag{2.45}$$

计算升交角距的变化率：

$$\dot{\Phi}_k = \frac{\sqrt{1-e^2}}{1 - e\cos E_k}\dot{E}_k \tag{2.46}$$

计算经过改正的升交角距的变化率：

$$\dot{u}_k = 2\left(C_{us}\cos 2\Phi_k - C_{uc}\sin 2\Phi_k\right)\cdot\dot{\Phi}_k + \dot{\Phi}_k \tag{2.47}$$

计算经过改正的向径的变化率：

$$\dot{r}_k = Ae\sin\left(E_k\right)\dot{E}_k + 2\left(C_{rs}\cos 2\Phi_k - C_{rc}\sin 2\Phi_k\right)\dot{\Phi}_k \tag{2.48}$$

计算经过改正的轨道倾角的变化率：

$$\dot{i}_k = IDOT + 2\left(C_{is}\cos 2\Phi_k - C_{ic}\sin 2\Phi_k\right)\dot{\Phi}_k \tag{2.49}$$

计算改正后的升交点经度的变化率：

$$\dot{\Omega}_k = \dot{\Omega} - \dot{\Omega}_e \tag{2.50}$$

构建 $\dot{R}$ 矩阵：

$$\dot{\mathrm{R}}=\begin{bmatrix} \cos\Omega_k & -\sin\Omega_k\cos i_k & -\left(x_k^{'}\sin\Omega_k+y_k^{'}\cos\Omega_k\cos i_k\right) & y_k^{'}\sin\Omega_k\sin i_k \\ \sin\Omega_k & \cos\Omega_k\cos i_k & \left(x_k^{'}\cos\Omega_k-y_k^{'}\sin\Omega_k\cos i_k\right) & -y_k^{'}\cos\Omega_k\sin i_k \\ 0 & \sin i_k & 0 & y_k^{'}\cos i_k \end{bmatrix} \tag{2.51}$$

计算卫星在轨道平面上的位置的变化率：

$$\begin{cases} \dot{x}_k^{'}=\dot{r}_k\cos u_k-r_k\dot{u}_k\sin u_k \\ \dot{y}_k^{'}=\dot{r}_k\sin u_k+r_k\dot{u}_k\cos u_k \end{cases} \tag{2.52}$$

计算卫星速度：

$$\begin{bmatrix} \dot{x}_k \\ \dot{y}_k \\ \dot{z}_k \end{bmatrix}=\dot{\mathrm{R}}\begin{bmatrix} \dot{x}_k^{'} \\ \dot{y}_k^{'} \\ \dot{\Omega}_k \\ \dot{i}_k \end{bmatrix} \tag{2.53}$$

（3）GPS 卫星钟差

$$\Delta\mathrm{t}=\left(t_{obs}^{week}-t_{TOC}^{week}\right)\cdot 604800+\left(t_{obs}^{\sec}-t_{TOC}^{\sec}\right) \tag{2.54}$$

$$\Delta t_{SV}=ClkBias+ClkDrift\cdot\Delta\mathrm{t}+ClkDriftRate\cdot\Delta\mathrm{t}^2+\Delta t_r \tag{2.55}$$

$$\Delta t_r=Fe\sqrt{A}\sin E_k \tag{2.56}$$

$$F=\frac{-2\sqrt{u}}{c^2}=-4.442807633\cdot 10^{-10}\left(\sec\cdot m^{-1/2}\right) \tag{2.57}$$

其中，Δt_r 为相对论效应改正。

（4）GPS 卫星钟速

$$\frac{d\left(\Delta t_{SV}\right)}{dt}=ClkDrift+2\cdot ClkDriftRate\cdot\Delta\mathrm{t}+\Delta\dot{t}_r \tag{2.58}$$

$$\Delta\dot{t}_r=Fe\sqrt{A}\cos E_k\dot{E}_k \tag{2.59}$$

其中：$\Delta\dot{t}_r$ 为相对论效应改正的导数。

2.5.3.2 函数设计

根据 GPS 卫星位置、速度、钟差和钟速的算法原理，程序中的函数设计如表 2.21：

表 2.21 GPS 卫星计算函数设计说明

函数	内容
名称	CompGPSSatPVT
声明	bool CompGPSSatPVT(const int Prn, const GPSTIME* t, const GPSEPHREC* Eph, SATMIDRES* Mid);
功能	计算 GPS 卫星位置、速度、钟差、钟速
输入	Prn 号，卫星信号发射时刻，卫星星历数据
输出	GPS 卫星位置、速度、钟差、钟速

2.5.4 BDS 卫星计算函数

2.5.4.1 算法原理

（1）BDS 卫星位置

BDS 系统下的地球重力常数 $\mu = 3.986004418 \times 10^{14} m^3 / s^2$，地球自转角速度 $\dot{\Omega}_e = 7.2921150 \times 10^{-5} rad / s$ 。

在计算 BDS 卫星位置前，需要将运算所使用的 GPS 时转换成 BDS 时：

$$GPST - BDT = 14s \tag{2.60}$$

$$GPS\ WEEK - BDS\ WEEK = 1356 \tag{2.61}$$

北斗 MEO 和 IGSO 卫星位置的计算方法和 GPS 卫星位置计算方法相同，而同步轨道 GEO 卫星的计算方法则有所区别。首先需要判断该卫星是否属于 GEO 卫星，有以下两种方法：

①根据 PRN 识别，GEO 卫星的 PRN 范围为[1,5]、[59,63]。

②根据轨道倾角 i_0，MEO 和 IGSO 卫星的轨道倾角约为 55°，GEO 则为 0°，在程序中可以用 $i_0 < 30^\circ$ 来识别 GEO。

GEO 卫星与 MEO/IGSO 卫星位置计算的区别如下：

$$\Omega_k = \Omega_0 + \dot{\Omega} t_k - \dot{\Omega}_e t_{oe} \tag{2.62}$$

$$\begin{cases} X_{GK} = x_k \cos\Omega_k - y_k \cos i_k \sin\Omega_k \\ Y_{GK} = x_k \sin\Omega_k + y_k \cos i_k \cos\Omega_k \\ Z_{GK} = y_k \sin i_k \end{cases} \tag{2.63}$$

$$\begin{bmatrix} X_k \\ Y_k \\ Z_k \end{bmatrix} = R_Z\left(\dot{\Omega}_e t_k\right) R_X\left(-5^\circ\right) \begin{bmatrix} X_{GK} \\ Y_{GK} \\ Z_{GK} \end{bmatrix} \tag{2.64}$$

其中：

$$R_X(\varphi) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos\varphi & \sin\varphi \\ 0 & -\sin\varphi & \cos\varphi \end{pmatrix} \tag{2.65}$$

$$R_Z(\varphi) = \begin{pmatrix} \cos\varphi & \sin\varphi & 0 \\ -\sin\varphi & \cos\varphi & 0 \\ 0 & 0 & 1 \end{pmatrix} \tag{2.66}$$

（2）BDS 卫星速度

北斗 MEO 和 IGSO 卫星速度的计算方法和 GPS 卫星位置计算方法相同，而同步轨道 GEO 卫星的计算方法则有所区别。GEO 卫星与 MEO/IGSO 卫星位置计算的区别如下：

$$\dot{\Omega}_k = \dot{\Omega} \tag{2.67}$$

$$\frac{d}{dt}\begin{bmatrix} X_k \\ Y_k \\ Z_k \end{bmatrix} = R_Z\left(\dot{\Omega}_e t_k\right) R_X\left(-5^\circ\right) \frac{d}{dt}\begin{bmatrix} X_{GK} \\ Y_{GK} \\ Z_{GK} \end{bmatrix} + \dot{R}_Z\left(\dot{\Omega}_e t_k\right) R_X\left(-5^\circ\right) \begin{bmatrix} X_{GK} \\ Y_{GK} \\ Z_{GK} \end{bmatrix} \tag{2.68}$$

（3）BDS 卫星钟差和钟速

同 GPS，但需注意地球自转角速度需使用 CGCS-2000 坐标系下定义的。

2.5.4.2 函数设计

根据 BDS 卫星位置、速度、钟差和钟速的算法原理，程序中的函数设计如表 2.22：

表 2.22 BDS 卫星计算函数设计说明

函数	内容
名称	CompBDSSatPVT
声明	bool CompBDSSatPVT(const int Prn, const GPSTIME* t, const GPSEPHREC* Eph, SATMIDRES* Mid);
功能	计算 BDS 卫星位置、速度、钟差、钟速
输入	Prn 号，卫星信号发射时刻，卫星星历数据
输出	BDS 卫星位置、速度、钟差、钟速

2.5.5 地球自转改正函数

2.5.5.1 算法原理

卫星的位置和速度需要根据接收机的位置进行地球自转改正：

$$X = R_Z\left(\omega_e \Delta t\right)\begin{bmatrix} x_k \\ y_k \\ z_k \end{bmatrix} \tag{2.69}$$

$$\dot{X} = R_Z\left(\omega_e \Delta t\right)\begin{bmatrix} \dot{x}_k \\ \dot{y}_k \\ \dot{z}_k \end{bmatrix} \tag{2.70}$$

其中，ω_e 为地球自转角速度，Δt 为信号传播时间：

$$\rho = \sqrt{\left(x^s - x_r\right)^2 + \left(y^s - y_r\right)^2 + \left(z^s - z_r\right)^2} \tag{2.71}$$

$$\Delta t = \frac{\rho}{c} \tag{2.72}$$

2.5.5.2 函数设计

程序中，地球自转改正函数设计如表 2.23：

表 2.23 地球自转改正函数设计说明

函数	内容
名称	RotationCorrection

声明	`void RotationCorrection(SATMIDRES* Mid, double UserPos[3], double Omega_e);`
功能	地球自转改正
输入	卫星的位置和速度，用户位置，自转角速度
输出	经地球自转改正后的卫星位置和速度

2.6 误差改正

2.6.1 大气误差

2.6.1.1 对流层延迟改正算法

程序采用 Hopefield 模型进行对流层延迟改正，计算公式如下：

$$h_d = 40136 + 148.72\left(T_0 - 273.16\right) \tag{2.73}$$

$$h_w = 11000m \tag{2.74}$$

$$e = RH \cdot \exp\left(-37.2465 + 0.213166T - 0.000256908T^2\right) \tag{2.75}$$

$$T = T_0 - 0.0065 \cdot \left(H - H_0\right) \tag{2.76}$$

$$p = p_0 \cdot \left(1 - 0.0000226 \cdot \left(H - H_0\right)\right)^{5.225} \tag{2.77}$$

$$RH = RH_0 \cdot \exp\left(-0.0006396 \cdot \left(H - H_0\right)\right) \tag{2.78}$$

$$K_d = 155.2 \cdot 10^{-7} \cdot \frac{p}{T}\left(h_d - H\right) \tag{2.79}$$

$$K_w = 155.2 \cdot 10^{-7} \cdot \frac{4810}{T^2} e\left(h_w - H\right) \tag{2.80}$$

$$\Delta_{Trop} = \Delta_d + \Delta_w = \frac{K_d}{\sin\sqrt{E^2 + 6.25^\circ}} + \frac{K_w}{\sin\sqrt{E^2 + 2.25^\circ}} \tag{2.81}$$

其中：H 为测站高度，T、p 和 RH 分别为参考面上的干温、气压和相对湿度；E 为卫星相对于测站的高度角，单位为角度。

模型所使用的标准气象元素如下：海平面 $H_0 = 0m$；温度 $T_0 = 15^\circ\text{C}+273.16$；气压 $p_0 = 1013.25mbar$；相对湿度 $RH_0 = 50\%$。

2.6.1.2 电离层延迟改正算法

本程序采用双频消电离层组合来处理电离层延迟。

伪距观测方程：

$$P_{f_1}^j(k) = \rho^j(k) + c \cdot dt_R(k) - c \cdot dt^j(k) + I_{f_1}^j(k) + Tro^j(k) + \varepsilon_P \tag{2.82}$$

$$P_{f_2}^j(k) = \rho^j(k) + c \cdot dt_R(k) - c \cdot dt^j(k) + I_{f_2}^j(k) + Tro^j(k) + \varepsilon_P \tag{2.83}$$

相位观测方程：

$$\lambda_{f_1} \cdot \left[N_{f_1} + \varphi_{f_1}^j(k)\right] = \rho^j(k) + c \cdot dt_R(k) - c \cdot dt^j(k) - I_{f_1}^j(k) + Tro^j(k) + \varepsilon_\Phi \tag{2.84}$$

$$\lambda_{f_2} \cdot \left[N_{f_2} + \varphi_{f_2}^j(k)\right] = \rho^j(k) + c \cdot dt_R(k) - c \cdot dt^j(k) - I_{f_2}^j(k) + Tro^j(k) + \varepsilon_\Phi \tag{2.85}$$

相位 IF 组合（周为单位）：

$$\varphi_{IF} = \frac{f_{L1}^2}{f_{L1}^2 - f_{L2}^2}\varphi_1 - \frac{f_{L1} \cdot f_{L2}}{f_{L1}^2 - f_{L2}^2}\varphi_2 \tag{2.86}$$

相位 IF 组合（米为单位）：

$$L_{IF} = \frac{1}{f_1^2 - f_2^2}\left(f_1^2 L_1 - f_2^2 L_2\right) \tag{2.87}$$

伪距 IF 组合：

$$P_{IF} = \frac{1}{f_1^2 - f_2^2}\left(f_1^2 P_1 - f_2^2 P_2\right) \tag{2.88}$$

2.6.1.3 大气误差函数设计

程序中，电离层延迟改正在粗差探测函数中实现，对流层延迟改正函数设计如表 2.24：

表 2.24 对流层延迟改正函数设计说明

函数	内容
名称	hopfield
声明	`double hopfield(double hgt, double elev);`
功能	对流层延迟改正
输入	测站高度、高度角
输出	返回对流层改正值
容错处理	测站高度不在对流层范围内，直接输出为 0

2.6.2 粗差探测

2.6.2.1 算法原理

本程序中使用 MW 组合观测值和电离层残差组合观测值来进行粗差探测。

MW 组合观测值：

$$L_{MW}=\frac{1}{f_1-f_2}\left(f_1L_1-f_2L_2\right)-\frac{1}{f_1+f_2}\left(f_1P_1-f_2P_2\right) \tag{2.89}$$

在历元间 MW 组合观测值的平滑值计算方式如下：

$$\bar{L}_{MW}\left(t_k\right)=\frac{\left(k-1\right)\times\bar{L}_{MW}\left(t_{k-1}\right)+L_{MW}\left(t_k\right)}{k} \tag{2.90}$$

电离层残差组合观测值：

$$L_{GF}=L_1-L_2=\lambda_1\varphi_1-\lambda_2\varphi_2 \tag{2.91}$$

计算当前历元与前一历元 MW 组合观测值和电离层残差组合观测值之间的限差，与阈值进行比较，MW 组合观测值的阈值设置为 3m，电离层残差组合观测值的阈值设置为 5cm。若超限则标记为粗差：

$$\left|L_{GF}\left(t_k\right)-L_{GF}\left(t_{k-1}\right)\right|<\delta_{GF} \tag{2.92}$$

$$\left|L_{MW}\left(t_k\right)-\bar{L}_{MW}\left(t_{k-1}\right)\right|<\delta_{MW} \tag{2.93}$$

2.6.2.2 结构体定义

定义存储粗差探测数据的结构体：

```
struct MWGF
{
 short Prn;//卫星号
 GNSSSys Sys;
 double MW;
 double GF;
 double PIF;
 int n; //平滑计数

 MWGF()
 {
  Prn = n = 0;
  Sys = UNKS;
  MW = GF = PIF = 0.0;
 }
};
```

```
//定义每个历元的观测数据
struct EPOCHOBSDATA {
 GPSTIME Time;      //从头文件中获取
 short SatNum;
 SATOBSDATA SatObs[MAXCHANNUM];
 MWGF ComObs[MAXCHANNUM];
 SATMIDRES  SatPVT[MAXCHANNUM];

 EPOCHOBSDATA()
 {
  SatNum = 0;
 }
};
```

2.6.2.3 函数设计

函数内部先定义 MWGF CurComObs[MAXCHANNUM]，用于存放当前历元的计算结果，EPOCHOBSDATA 结构体中 ComObs 保存上个历元的平滑结果。

对于任意一颗卫星的观测数据，其粗差探测的步骤为：

（1）检查该卫星的双频伪距和相位数据是否有效和完整，若不全或为 0，

将 Valid 标记为 false，continue；

（2）计算当前历元该卫星的 GF 和 MW 组合值；

（3）从上个历元的 MWGF 数据中查找该卫星的 GF 和 MW 组合值；

（4）计算当前历元该卫星 GF 与上一历元对应 GF 的差值 dGF；

（5）计算当前历元该卫星 MW 与上一历元对应 MW 平滑值的差值 dMW；

（6）检查 dGF 和 dMW 是否超限，限差阈值建议为 5cm 和 3m。若超限，标记为粗差，将 Valid 标记为 false ，若不超限，标记为可用将 Valid 标记为 true，并计算该卫星的 MW 平滑值；

（7）对于可用的观测数据，计算伪距的 IF 组合观测值，用于 SPP；

（8）所有卫星循环计算完成之后，将 CurComObs 内存拷贝到 ComObs，即函数运行结束后， ComObs 保存了当前历元的 GF 和 MW 平滑值。

程序中，粗差探测函数设计如表 2.25：

表 2.25 粗差探测函数设计说明

函数	内容
名称	DetectOutlier
声明	void DetectOutlier(EPOCHOBSDATA* obs);
功能	对流层延迟改正
输入	观测值
输出	组合观测值

2.7 单点定位与测速

2.7.1 单点定位

2.7.1.1 算法原理

GPS 和 BDS 的伪距观测方程如下：

$$P_{GL_i}^j = \rho_G^j + c \cdot \delta t_G - c \cdot \delta t_G^j + I_{GL_i}^j + T^j + Tgd_{GL_i}^j + \varepsilon_{GL_i}^j \tag{2.93}$$

$$P_{CB_i}^k = \rho_C^k + c \cdot \delta t_C - c \cdot \delta t_C^k + I_{CB_i}^k + T^k + Tgd_{CB_i}^k + \varepsilon_{CB_i}^k \tag{2.94}$$

本程序使用双频消电离层组合观测值来进行定位解算，所以双频无电离层组合的伪距观测方程如下：

$$P_{G,IF}^{j}=\rho_{G}^{j}+c\cdot\delta t_{G}-c\cdot\delta t_{G}^{j}+T^{j}+Tgd_{G,IF}^{j}+\varepsilon_{G,IF}^{j} \tag{2.95}$$

$$P_{C,IF}^{j}=\rho_{C}^{j}+c\cdot\delta t_{C}-c\cdot\delta t_{C}^{j}+T^{j}+Tgd_{C,IF}^{j}+\varepsilon_{C,IF}^{j} \tag{2.96}$$

考虑到在进行定位解算的时候所使用的卫星位置是信号发射时刻的卫星位置，所以考虑观测历元时刻的观测方程为：

$$P_{G,IF}^{j}\left(t_{R}\right)=\rho_{G}^{j}\left(t_{R,GPST},t_{R,GPST}-\Delta t\right)+c\cdot\delta t_{G}\left(t_{R,GPST}\right)-c\cdot\delta t_{G}^{j}\left(t_{R,GPST}-\Delta t\right)+T^{j}+\varepsilon_{G,IF}^{j} \tag{2.97}$$

$$P_{C,IF}^{j}\left(t_{R}\right)=\rho_{C}^{j}\left(t_{R,BDT},t_{R,BDT}-\Delta t\right)+c\cdot\delta t_{C}\left(t_{R,BDT}\right)-c\cdot\delta t_{C}^{j}\left(t_{R,BDT}-\Delta t\right)+T^{j}+Tgd_{C,IF}^{j}+\varepsilon_{C,IF}^{j} \tag{2.98}$$

$$\rho_{G}^{j}\left(t_{R,GPST},t_{R,GPST}-\Delta t\right)=\left\|X_{G}^{j}\left(t_{R,GPST}-\Delta t\right)-X\left(t_{R,GPST}\right)\right\| \tag{2.99}$$

$$\rho_{C}^{j}\left(t_{R,BDT},t_{R,BDT}-\Delta t\right)=\left\|X_{C}^{j}\left(t_{R,BDT}-\Delta t\right)-X\left(t_{R,BDT}\right)\right\| \tag{2.100}$$

其中，t_R 为观测历元时刻，也就是信号接收机表面时；$t_{R,GPST}$ 为该时刻的 GPS 时；$t_{R,BDT}$ 为该时刻的北斗时；Δt 为信号传播时间；$\rho_{G}^{j}\left(t_{R,GPST},t_{R,GPST}-\Delta t\right)$ 为历元 t_R 时刻测站与卫星 j 之间的几何距离；$\delta t_{G}^{j}\left(t_{R,GPST}-\Delta t\right)$ 为信号发射时刻的 GPS 卫星钟差，以秒为单位；$\delta t_{G}\left(t_{R,GPST}\right)$ 为信号接收时刻的接收机钟差，以秒为单位；

设置用户位置的初值 $\vec{X}_{R}^{0}$，对观测方程进行泰勒级数展开，保留一阶项：

$$\begin{aligned}&P_{i,IF}^{j}\left(t\right)+v_{i,IF}^{j}\left(t\right)=\rho_{i}^{j0}\left(t_{R,i},t_{R,i}-\Delta t\right)+\delta t_{i}^{0}\left(t_{R,i}\right)-c\cdot\delta t_{i}^{j}\left(t_{R,i}-\Delta t\right)+T^{j}+Tgd_{i,IF}^{j}\\&+\frac{X_{R}^{0}-X_{i}^{j}\left(t_{R,i}-\Delta t\right)}{\rho_{i}^{j0}\left(t_{R,i},t_{R,i}-\Delta t\right)}x_{R}+\frac{Y_{R}^{0}-Y_{i}^{j}\left(t_{R,i}-\Delta t\right)}{\rho_{i}^{j0}\left(t_{R,i},t_{R,i}-\Delta t\right)}y_{R}+\frac{Z_{R}^{0}-Z_{i}^{j}\left(t_{R,i}-\Delta t\right)}{\rho_{i}^{j0}\left(t_{R,i},t_{R,i}-\Delta t\right)}z_{R}+\delta t_{i}\left(t_{R,i}\right)\end{aligned} \tag{2.101}$$

$$\begin{aligned}&v_{i,IF}^{j}\left(t\right)=\frac{X_{R}^{0}-X_{i}^{j}\left(t_{R,i}-\Delta t\right)}{\rho_{i}^{j0}\left(t_{R,i},t_{R,i}-\Delta t\right)}x_{R}+\frac{Y_{R}^{0}-Y_{i}^{j}\left(t_{R,i}-\Delta t\right)}{\rho_{i}^{j0}\left(t_{R,i},t_{R,i}-\Delta t\right)}y_{R}+\frac{Z_{R}^{0}-Z_{i}^{j}\left(t_{R,i}-\Delta t\right)}{\rho_{i}^{j0}\left(t_{R,i},t_{R,i}-\Delta t\right)}z_{R}+\delta t_{i}\left(t_{R,i}\right)\\&-\left\{P_{i,IF}^{j}\left(t\right)-\left[\rho_{i}^{j0}\left(t_{R,i},t_{R,i}-\Delta t\right)+\delta t_{i}^{0}\left(t_{R,i}\right)-c\cdot\delta t_{i}^{j}\left(t_{R,i}-\Delta t\right)+T^{j}+Tgd_{i,IF}^{j}\right]\right\}\end{aligned} \tag{2.102}$$

其中，$\rho_{i}^{j0}\left(t_{R,i},t_{R,i}-\Delta t\right)$ 是根据用户位置初值 $\vec{X}_{R}^{0}$ 计算得到的几何距离：

$$\vec{X}_{R}=\vec{X}_{R}^{0}+\vec{x}_{R} \tag{2.103}$$

若令：$l_{i}^{j}\left(t\right)=\dfrac{X_{R}^{0}-X_{i}^{j}\left(t_{R,i}-\Delta t\right)}{\rho_{i}^{j0}\left(t_{R,\mathrm{i}},t_{R,\mathrm{i}}-\Delta t\right)}$，$m_{i}^{j}\left(t\right)=\dfrac{Y_{R}^{0}-Y_{i}^{j}\left(t_{R,i}-\Delta t\right)}{\rho_{i}^{j0}\left(t_{R,\mathrm{i}},t_{R,\mathrm{i}}-\Delta t\right)}$，$n_{i}^{j}\left(t\right)=\dfrac{Z_{R}^{0}-Z_{i}^{j}\left(t_{R,i}-\Delta t\right)}{\rho_{i}^{j0}\left(t_{R,\mathrm{i}},t_{R,\mathrm{i}}-\Delta t\right)}$

$w_{i}^{j}\left(t\right)=P_{i,IF}^{j}\left(t\right)-\left[\rho_{i}^{j0}\left(t_{R,i},t_{R,i}-\Delta t\right)+\delta t_{i}^{0}\left(t_{R,i}\right)-c\cdot\delta t_{i}^{j}\left(t_{R,i}-\Delta t\right)+T^{j}+Tgd_{i,IF}^{j}\right]$，则：

$$v_{i,IF}^{j}(t)=l_i^j(t)x_R+m_i^j(t)y_R+n_i^j(t)z_R+\delta t_G\left(t_{R,GPST}\right)+\delta t_C\left(t_{R,BDT}\right)-w_i^j(t) \quad (2.104)$$

接着进行最小二乘解运算，由于对观测方程进行线性化，结算过程中需要进行迭代更新：

$$x=\left(\underset{n\times p}{B^T}\underset{n\times n}{P}\underset{n\times p}{B}\right)^{-1}\underset{n\times p}{B^T}\underset{n\times n}{P}w \quad (2.105)$$

其中若仅有 GPS 卫星星历时，裁剪 B 的第 5 列，此时列数为 4；若仅有 BDS 卫星星历时，裁剪 B 的第 4 列，此时列数为 4；若同时有 GPS 与 BDS 卫星星历，不裁剪 B 的列数，此时列数为 5。

对定位精度进行评估：

$$\hat{\sigma}_0=\sqrt{\frac{v^TPv}{s-5}} \quad (2.106)$$

$$Q_{\hat{x}\hat{x}}=\left(B^TPB\right)^{-1}=\begin{bmatrix} q_{\hat{x}\hat{x}} & q_{\hat{x}\hat{y}} & q_{\hat{x}\hat{z}} & q_{\hat{x}\delta\hat{t}_G} & q_{\hat{x}\delta\hat{t}_C} \\ q_{\hat{y}\hat{x}} & q_{\hat{y}\hat{y}} & q_{\hat{y}\hat{z}} & q_{\hat{y}\delta\hat{t}_G} & q_{\hat{y}\delta\hat{t}_C} \\ q_{\hat{z}\hat{x}} & q_{\hat{z}\hat{y}} & q_{\hat{z}\hat{z}} & q_{\hat{z}\delta\hat{t}_G} & q_{\hat{z}\delta\hat{t}_C} \\ q_{\delta\hat{t}_G\hat{x}} & q_{\delta\hat{t}_G\hat{y}} & q_{\delta\hat{t}_G\hat{z}} & q_{\delta\hat{t}_G\delta\hat{t}_G} & q_{\delta\hat{t}_G\delta\hat{t}_C} \\ q_{\delta\hat{t}_C\hat{x}} & q_{\delta\hat{t}_C\hat{y}} & q_{\delta\hat{t}_C\hat{z}} & q_{\delta\hat{t}_C\delta\hat{t}_G} & q_{\delta\hat{t}_C\delta\hat{t}_C} \end{bmatrix} \quad (2.107)$$

$$W=B^TPw=\begin{bmatrix} w_{\hat{x}} \\ w_{\hat{y}} \\ w_{\hat{z}} \\ w_{\delta\hat{t}_G} \\ w_{\delta\hat{t}_C} \end{bmatrix} \quad (2.108)$$

$$PDOP=\sqrt{q_{\hat{x}\hat{x}}+q_{\hat{y}\hat{y}}+q_{\hat{z}\hat{z}}} \quad (2.109)$$

2.7.1.2 单点定位求解函数

单点定位求解函数的流程设计如图 2.8，其中将观测方程线性化、判断卫星个数、最小二乘求解和精度评定通过调用最小二乘函数实现。

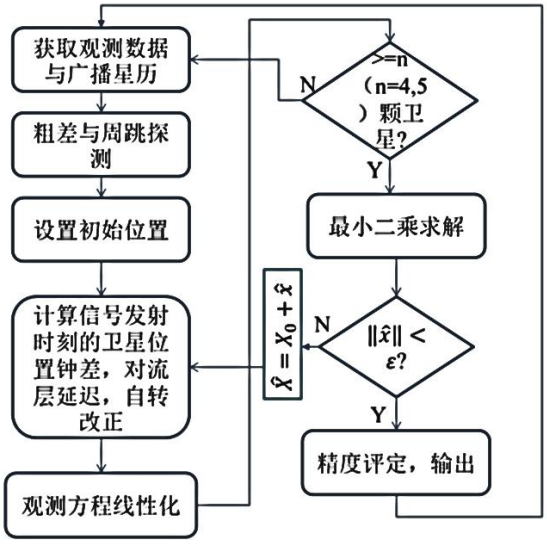

图 2.8 单点定位求解函数的流程设计

程序中，单点定位求解函数设计如表 2.26：

表 2.26 单点定位求解函数设计说明

函数	内容
名称	SPP
声明	bool SPP(EPOCHOBSDATA* Epoch, GPSEPHREC* GPSEph, GPSEPHREC* BDSEph, POSRES* Res);
功能	单点定位
输入	观测数据的结构体，卫星星历
输出	用户定位结果
容错处理	少于 4 颗卫星不进行定位解算

2.7.1.3 最小二乘函数

定义最小二乘函数，实现单点定位中的观测方程线性化、判断卫星个数、最小二乘求解和精度评定，函数设计如表 2.27：

表 2.27 最小二乘函数设计说明

函数	内容
名称	SPP_LS
声明	bool SPP_LS(EPOCHOBSDATA* obs, POSRES* Res);

功能	实现最小二乘
输入	观测数据
输出	用户定位结果
容错处理	少于 4 颗卫星不进行定位解算

2.7.2 单点测速

2.7.2.1 算法原理

观测方程：

$$D_i^j(t)+v_i^j(t)=\dot{\rho}_R^j(t,t-\Delta t)+\delta\dot{t}_i(t)-c\cdot\delta\dot{t}^j(t-\Delta t)+\dot{d}_{itrop}^j(t)+\dot{d}_{iion}^j(t) \tag{2.110}$$

$$\dot{\rho}_R^j(t,t-\Delta t)=\frac{\left(X^j-X_R\right)\left(\dot{X}^j-\dot{X}_R\right)+\left(Y^j-Y_R\right)\left(\dot{Y}^j-\dot{Y}_R\right)+\left(Z^j-Z_R\right)\left(\dot{Z}^j-\dot{Z}_R\right)}{\rho_R^j(t,t-\Delta t)} \tag{2.111}$$

$$\rho_R^j(t,t-\Delta t)=\sqrt{\left(X^j(t-\Delta t)-X_R\right)^2+\left(Y^j(t-\Delta t)-Y_R\right)^2+\left(Z^j(t-\Delta t)-Z_R\right)^2} \tag{2.112}$$

其中，$D_i^j(t)$ 为历元 t 时刻测站 R 对于卫星 j 频率 i 的多普勒观测值，以 m/s 为单位；$v_i^j(t)$ 为其改正数；$\delta\dot{t}_i(t)$ 为接收机钟差变化率，以 m/s 为单位；$\delta\dot{t}^j(t-\Delta t)$ 为卫星钟差变化率；$\dot{d}_{itrop}^j(t)$ 为对流层延迟变化率，单位 m/s；$\dot{d}_{iion}^j(t)$ 为电离层延迟变化率，单位 m/s。

单点定位计算成功后，得到 X_R,测速观测方程整理得到：

$$\begin{aligned}D_1^j(t)+v_1^j(t)=&\dot{\rho}^j(t,t-\Delta t)+\frac{x_R-x^j(t-\Delta t)}{\rho^j(t,t-\Delta t)}\dot{x}_R+\frac{y_R-y^j(t-\Delta t)}{\rho^j(t,t-\Delta t)}\dot{y}_R+\frac{z_R-z^j(t-\Delta t)}{\rho^j(t,t-\Delta t)}\dot{z}_R\\&+\delta\dot{t}(t)-c\cdot\delta\dot{t}^j(t-\Delta t)+\dot{d}_{itrop}^j(t)+\dot{d}_{iion}^j(t)\end{aligned} \tag{2.113}$$

$$\dot{\rho}^j(t,t-\Delta t)=\frac{\left(x^j-x_R\right)\dot{x}^j+\left(y^j-y_R\right)\dot{y}^j+\left(z^j-z_R\right)\dot{z}^j}{R^j(t,t-\Delta t)} \tag{2.114}$$

$$\rho^j(t,t-\Delta t)=\sqrt{\left(x^j(t-\Delta t)-x_R\right)^2+\left(y^j(t-\Delta t)-y_R\right)^2+\left(z^j(t-\Delta t)-z_R\right)^2} \tag{2.115}$$

若令：$l^j(t)=\dfrac{x_R-x^j(t-\Delta t)}{\rho^j(t,t-\Delta t)}$，$m^j(t)=\dfrac{y_R-y^j(t-\Delta t)}{\rho^j(t,t-\Delta t)}$，$n^j(t)=\dfrac{z_R-z^j(t-\Delta t)}{\rho^j(t,t-\Delta t)}$，

$w^j(t)=D_1^j(t)-\left(\dot{\rho}^j(t,t-\Delta t)-c\cdot\delta\dot{t}^j(t-\Delta t)\right)$，则：

$$v^j(t)=l^j(t)x_i+m^j(t)y_i+n^j(t)z_i+\delta\dot{t}(t)-w^j(t) \tag{2.116}$$

假定在某历元共对 s 颗卫星进行了观测，有：

$$v=BX-w \tag{2.117}$$

$$v=\begin{bmatrix} v^1 & v^2 & \cdots & v^s \end{bmatrix}^T \tag{2.118}$$

$$B=\begin{bmatrix} l^1 & m^1 & n^1 & 1 \\ l^2 & m^2 & n^2 & 1 \\ \cdots & \cdots & \cdots & \cdots \\ l^s & m^s & n^s & 1 \end{bmatrix} \tag{2.119}$$

$$X=\begin{bmatrix} \dot{x}_R & \dot{y}_R & \dot{z}_R & \delta\dot{\rho} \end{bmatrix}^T \tag{2.119}$$

$$w=\begin{bmatrix} w^1 & w^2 & \cdots & w^s \end{bmatrix}^T \tag{2.120}$$

$$\delta\dot{\rho}=c\cdot\delta\dot{t}(t) \tag{2.121}$$

观测值权阵为 P，则有：

$$\hat{X}=\left(B^TPB\right)^{-1}BPw \tag{2.122}$$

$$\hat{\sigma}_0=\sqrt{\frac{v^TPv}{s-4}} \tag{2.123}$$

速度的观测方程是线性方程组，不需要迭代计算。

2.7.2.2 函数设计

单点测速函数的流程设计如图 2.9，利用多普勒数据和单点定位解算得到的卫星位置、速度、钟差和钟速，构建矩阵与向量，进而进行最小二乘求解。

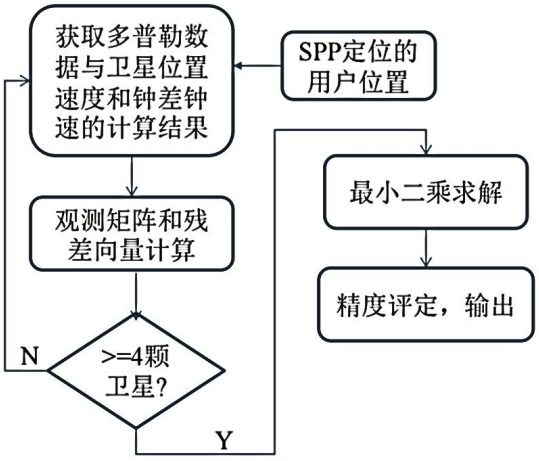

图 2.9 单点测速函数的流程设计

程序中，单点测速函数设计如表 2.28：

表 2.28 单点测速函数设计说明

函数	内容
名称	SPV
声明	bool SPV(EPOCHOBSDATA* obs, POSRES* Res);
功能	单点测速
输入	观测数据的结构体
输出	用户定位结果
容错处理	少于 4 颗卫星不进行定位解算

2.8 获取实时数据流

模块主要用来处理与套接字相关的操作，包括 OpenSocket()和 CloseSocket()两个函数。OpenSocket()函数用于创建一个 TCP 套接字并连接到指定的 IP 地址和端口；CloseSocket()函数用于关闭套接字并清理 Windows 套接字 API 的资源。

在主函数中调用这两个函数实现与网络串口的连接和断开与网络串口的连接。函数设计如表 2.29。

表 2.29 获取实时数据流函数设计说明

函数	内容 1	内容 2
名称	OpenSocket	CloseSocket
声明	bool OpenSocket(SOCKET_HANDLE*	void CloseSocket(SOCKET_HANDLE

	sock_handle, const char* ip, int port);	sock_handle);
功能	打开串口通信	关闭串口通信

2.9 主函数

主函数通过对各个模块的调用，实现单点解算并且输出。程序支持两种模式，通过键盘输入 0 或 1 进行选择，0 表示事后读取二进制文件进行数据处理，1 表示实时读取网路串口数据进行数据处理。函数设计思路如下：

（1）准备存储观测数据和解算结果的结构体。

（2）用户进行模式选择：选择 0，则打开二进制文件，使用循环逐块读取文件内容到缓冲区；选择 1，建立与 GNSS 基站 (47.114.134.129:7190) 的 socket 连接，通过 recv 函数循环接收数据流。

（3）调用解码主函数 DecodeNovOem7Dat() 对主缓冲区中的二进制数据流进行解析。

（4）解码成功并获得一个历元的完整观测数据后，调用 DetectOutlier() 函数，探测并标记潜在的粗差和周跳，确保后续解算的可靠性。

（5）调用 SPP() 函数，利用经过粗差探测的观测数据和已获取的广播星历，进行标准单点定位解算。

（6）若 SPP() 解算成功，调用 SPV() 函数，利用多普勒观测值进行单点测速解算。

（7）在文件模式下，将卫星信息和定位结果写入 “output_file.txt”；在串口网络模式下，调用 Comp_dEnu() 计算 ENU 坐标系下的位置偏差，将关键定位结果实时打印到控制台同时将精简结果写入 “output_net.txt”，用于用户使用或者后续精度分析。

三、结果精度分析

3.1 结果展示

程序实时采集了一个时段的数据进行精度分析，时段如下：

表 3.1 数据采集时段

GPS 时	总时长
2025-07-28 16:59:26 至 2025-07-29 07:40:56	14.69

实时对采集数据进行定位计算得到接收机三维坐标位置后，使用解算结果与 BestPos 中播发的精确结果进行对比分析，将计算得到 ENU 方向的定位误差输出到“output_net.txt”中，并编写 matlap 程序对结果进行可视化。

```
SPP: 2377 147566.0000 GPS Clk= -0.066 BDS Clk= 6.737 RcvSft= 0.060 PDOP= 0.940 SigmaPos= 1.399 SigmaVel= 0.016 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.2243 dN= 0.2774 dU= 2.4835
SPP: 2377 147567.0000 GPS Clk= 0.056 BDS Clk= 6.860 RcvSft= 0.122 PDOP= 0.940 SigmaPos= 1.397 SigmaVel= 0.013 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.2100 dN= 0.2760 dU= 2.4994
SPP: 2377 147568.0000 GPS Clk= 0.149 BDS Clk= 6.973 RcvSft= 0.116 PDOP= 0.940 SigmaPos= 1.400 SigmaVel= 0.024 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.1989 dN= 0.2747 dU= 2.5286
SPP: 2377 147569.0000 GPS Clk= 0.280 BDS Clk= 7.108 RcvSft= 0.119 PDOP= 0.940 SigmaPos= 1.400 SigmaVel= 0.015 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.1729 dN= 0.2646 dU= 2.5492
SPP: 2377 147570.0000 GPS Clk= 0.422 BDS Clk= 7.251 RcvSft= 0.110 PDOP= 0.940 SigmaPos= 1.400 SigmaVel= 0.021 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.1696 dN= 0.2517 dU= 2.5546
SPP: 2377 147571.0000 GPS Clk= 0.510 BDS Clk= 7.347 RcvSft= 0.034 PDOP= 0.940 SigmaPos= 1.401 SigmaVel= 0.018 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.1654 dN= 0.2337 dU= 2.5793
SPP: 2377 147572.0000 GPS Clk= 0.549 BDS Clk= 7.391 RcvSft= 0.091 PDOP= 0.940 SigmaPos= 1.401 SigmaVel= 0.020 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.1544 dN= 0.2393 dU= 2.5884
SPP: 2377 147573.0000 GPS Clk= 0.572 BDS Clk= 7.414 RcvSft= 0.081 PDOP= 0.940 SigmaPos= 1.403 SigmaVel= 0.015 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.1453 dN= 0.2336 dU= 2.5694
SPP: 2377 147574.0000 GPS Clk= 0.597 BDS Clk= 7.446 RcvSft= 0.029 PDOP= 0.940 SigmaPos= 1.403 SigmaVel= 0.021 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.1424 dN= 0.2241 dU= 2.5868
SPP: 2377 147575.0000 GPS Clk= 0.585 BDS Clk= 7.434 RcvSft= 0.001 PDOP= 0.940 SigmaPos= 1.405 SigmaVel= 0.015 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.1325 dN= 0.2082 dU= 2.5791
SPP: 2377 147576.0000 GPS Clk= 0.567 BDS Clk= 7.427 RcvSft= 0.005 PDOP= 0.940 SigmaPos= 1.405 SigmaVel= 0.017 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.1173 dN= 0.2073 dU= 2.6007
SPP: 2377 147577.0000 GPS Clk= 0.551 BDS Clk= 7.418 RcvSft= -0.056 PDOP= 0.940 SigmaPos= 1.405 SigmaVel= 0.018 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.1057 dN= 0.1988 dU= 2.6114
SPP: 2377 147578.0000 GPS Clk= 0.551 BDS Clk= 7.429 RcvSft= 0.054 PDOP= 0.940 SigmaPos= 1.406 SigmaVel= 0.022 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.0934 dN= 0.2011 dU= 2.6357
SPP: 2377 147579.0000 GPS Clk= 0.579 BDS Clk= 7.452 RcvSft= 0.041 PDOP= 0.940 SigmaPos= 1.405 SigmaVel= 0.013 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.0766 dN= 0.1890 dU= 2.6145
SPP: 2377 147580.0000 GPS Clk= 0.686 BDS Clk= 7.562 RcvSft= 0.129 PDOP= 0.940 SigmaPos= 1.405 SigmaVel= 0.028 GPSSats= 8 BDSSats= 20 Sats= 28 dE= -1.0574 dN= 0.1925 dU= 2.6361
```

图 3.1 “output_net.txt”部分输出结果

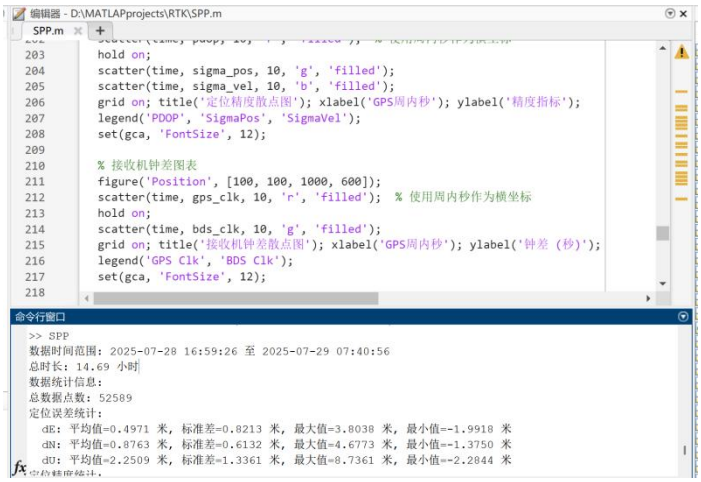

图 3.2 matlap 程序运行界面

3.2 精度分析

（1）定位误差分析

将实时定位结果与 BestPos 解码出的定位结果进行比较计算 ENU（东向、北向、天向）定位误差，结果可视化如下图：

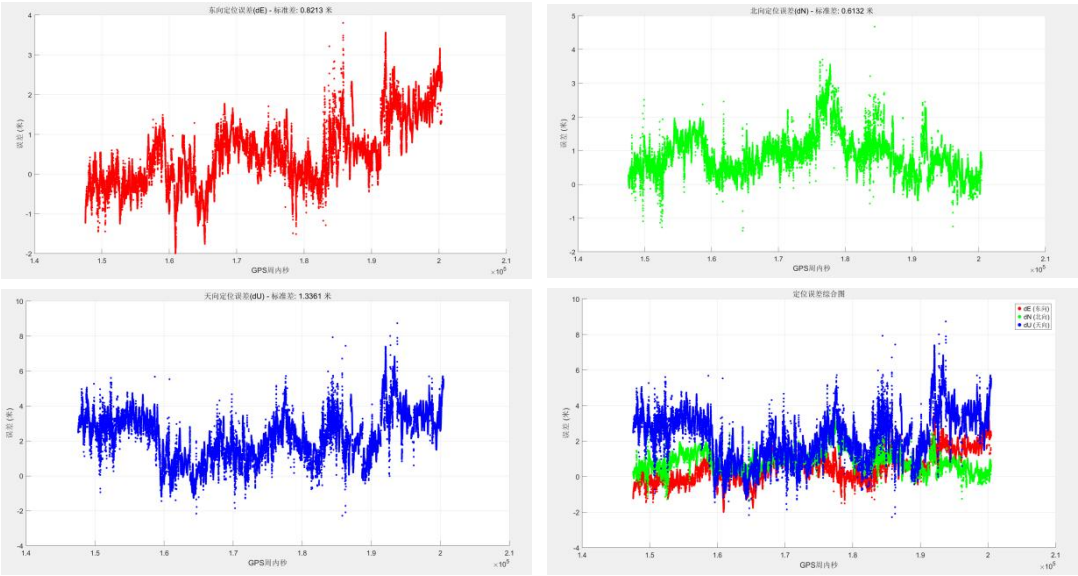

图 3.3 东向（左上） 北向（右上） 天向（左下） 综合（右下）定位误差散点图

表 3.2 定位误差统计表

属性	东向误差（m）	北向误差（m）	天向误差（m）
最小值	-1.9918	-1.3750	-2.2844
最大值	+3.8038	+4.6773	+8.7361
平均值	+0.4971	+0.8763	+2.2509
标准差	+0.8213	+0.6132	+1.3361

东（E）、北（N）、天（U）三个方向上的定位误差均值分别为+0.4971 m、+0.8763 m 和+2.2509 m。天向（U）分量的误差和偏差显著大于水平分量（E 和 N），这是 GNSS 定位的典型特征。由于所有可观测卫星均位于接收机地平线上方，缺乏来自下方的观测信息，导致垂直方向的几何约束较弱。这一固有的几何缺陷通过垂直精度因子（VDOP）进行量化，通常 VDOP 值会大于水平精度因子（HDOP），从而导致天向定位精度较低。

更深层次进行分析，E、N、U 三个方向上均存在一个持续的正向偏差。这种系统性偏差超出了零均值随机噪声的范畴。尽管部分原因可能源于参考基准（BestPos）坐标的微小偏移，但更可能的原因是算法本身的局限性。程序中所实现的标准最小二乘估计算法并未对观测值进行加权处理。这种处理方式赋予了所有卫星观测值同等的权重，但实际上，不同卫星的观测质量差异巨大。低高度角卫星的信号路径更长，受大气延迟和多路径效应的影响更为严重，其观测值噪声更大、可靠性更低。一个稳健的定位算法会通过加权策略降低这些低质量观测

值在解算中的贡献。当前算法由于缺乏加权，其定位结果极易被这些质量较差的观测值“拉偏”，从而引入一个持续的、系统性的位置偏差。

从图 3.3 还可以看出定位误差存在周期性。这种误差的周期性并非随机噪声，而是由两个核心物理现象驱动的系统性行为：卫星星座的几何构型重复性和空间相关的系统误差（如多路径效应）。GNSS 卫星在精确的轨道上运行，对于地面上的静态接收机而言，其观测到的卫星几何构型大约每隔一个恒星日（约 23 小时 56 分钟）会重复一次。如果接收机所处的观测环境中存在反射面（如建筑物、水体、车辆等），由信号反射引起的多路径误差也会随着卫星几何构型的重复而呈现出相似的周期性特征。此外，广播星历中的轨道和卫星钟差残余误差也包含周期性分量，是次要的误差来源。

（2）定位精度分析

将实时输出的 PDOP、SigmaPos、SigmaVel 进行可视化，结果如下图：

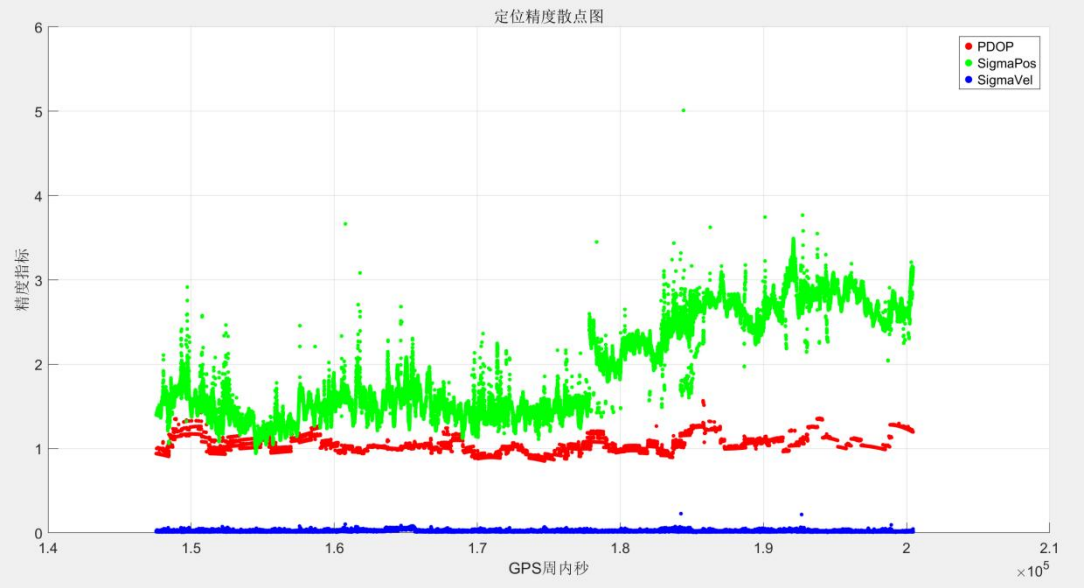

图 3.4 定位精度散点图

表 3.3 定位精度统计表

属性	SigmaPos（m）	SigmaVel（m/s）	PDOP
最小值	+0.9400	+0.0060	+0.8520
最大值	+5.0100	+0.2280	+1.5660
平均值	+1.9395	+0.0218	+1.0402
标准差	+0.5924	+0.0081	+0.0979,

从精度图可以看出，测试时间段内 PDOP 值较为稳定，SigmaPos 在周内秒 177881 后出现明显阶跃。

PDOP（位置精度因子）是一个纯粹的几何量，仅反映用于定位解算的卫星在空间中的几何分布强度。一个低且稳定的 PDOP 值（如观测到的平均值 1.0402，通常小于 2 即被认为是极佳的几何构型）表明在整个测试期间，卫星的几何布局非常理想，为高精度定位提供了良好的前提条件。

SigmaPos 是单位权中误差的验后估计值（σ_0）和位置解的协因数阵（其对角线元素与 PDOP 直接相关）的函数。在 PDOP 保持稳定的情况下，SigmaPos 的阶跃必然源于 σ_0的急剧增大。σ_0由伪距观测值的残差平方和（V^TPV）计算得出，其数值大小直接反映了所有观测值与解算出的几何模型之间的整体拟合程度。σ_0的突增意味着在那个时刻，原始伪距观测值的内部一致性突然显著恶化。这一现象的原因在于观测数据质量的急剧下降。可能的原因包括：(a) 一颗带有较大观测误差的新卫星被纳入了解算；(b) 周围环境发生变化（例如大型车辆经过），导致多颗卫星同时受到严重的多路径干扰；(c) 大气条件发生突变。由于当前算法采用未加权的最小二乘法，它无法有效抑制这些异常数据点的影响。一个质量极差的观测值被赋予了与十几个高质量观测值相同的权重，导致整体残差平方和急剧膨胀，进而使σ_0和 SigmaPos 发生阶跃。这不仅反映了数据质量的波动，也暴露了当前估计算法在面对异常数据时的脆弱性和缺乏稳健性。

（3）接收机钟差分析

将实时输出的 GPS Clk、BDS Clk 进行可视化，结果如下图：

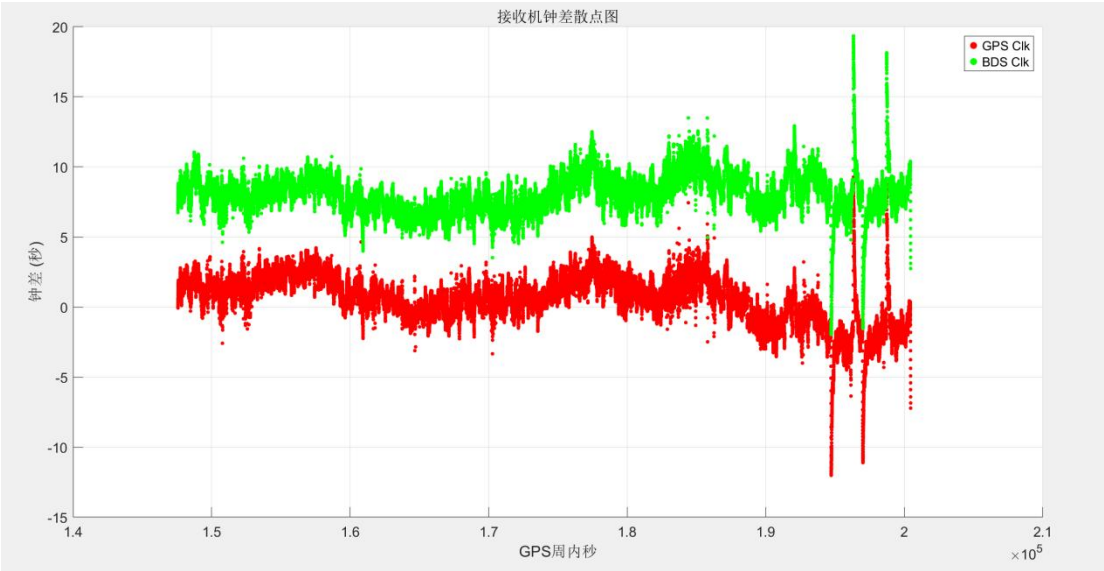

图 3.5 接收机钟差散点图

表 3.4 接收机钟差统计表

属性	GPS Clk	BDS Clk
最小值	-12.0130	-1.9550
最大值	+9.2690	+19.3440
平均值	+0.5665	+8.0806
标准差	+1.6944	+1.3880

观察可知，GPS 和 BDS 接收机钟差估计值在特定历元出现“明显的突变”，并且钟差存在“周期波动”。接收机内部通常使用成本较低的晶体振荡器，其频率稳定度远低于卫星上的原子钟，因此接收机钟差和钟漂会作为待估参数在导航解算中一并求出 。观测到的钟差跳变是接收机内部时钟驯服机制的典型行为。为保证解算过程中的数值稳定性，接收机固件会周期性地调整或重置本地时钟，使其相对于 GNSS 标准时间的偏差保持在一个预设的范围内（例如±1 毫秒） 。而钟差的周期性波动则通常与定位误差耦合在一起，当卫星位置、大气模型等存在残余误差时，这些误差的一部分会在最小二乘平差过程中被接收机钟差参数所吸收。

同时，GPS 和 BDS 钟差均值存在显著差异。软件为两个星座分别估计了独立的钟差参数（GPS Clk 和 BDS Clk） 。这两个均值之间的差异（约 7.5 ns）是对一个重要物理量——系统间偏差（Inter-System Bias, ISB）的直接量测。ISB 主要来源于两个方面：(a) 不同 GNSS 系统时间基准之间的微小差异（例如，GPST 与 BDT 并非完全同步）；(b) 接收机内部处理不同星座信号的硬件通道延迟差异 。该软件能够成功解算出稳定存在的 ISB，这从侧面验证了其双系统组合模型的正确性。然而，ISB 的稳定性对组合定位的性能至关重要。如果接收机内部的时钟跳变对 GPS 和 BDS 信号通道的影响不完全同步，可能会导致 ISB 在短时间内发生波动，从而对组合定位解的稳定性和精度造成瞬时影响。

3.3 优化方向

3.3.1 引入载波平滑伪距 (Hatch 滤波器)

伪距测量噪声较大，而载波相位测量虽然存在整周模糊度，但其变化量非常精确。载波平滑技术利用历元间精确的载波相位变化来平滑高噪声的伪距，相当于一个强大的低通滤波器，能够显著降低定位结果中的高频噪声。

实施递归的 Hatch 滤波器算法。在 k 历元的平滑伪距 $P_{\text{smooth}}(k)$ 是当前原始伪距 $P_{raw}(k)$ 与由载波相位变化递推过来的上一历元平滑伪距的加权平均：

$$P_{\text{smooth}}(k)=\frac{1}{N}P_{\text{raw}}(k)+\frac{N-1}{N}\left[P_{\text{smooth}}(k-1)+\lambda\left(\Phi(k)-\Phi(k-1)\right)\right] \tag{3.1}$$

其中，N 是平滑窗口长度，λ 是载波波长，Φ 是载波相位观测值。每当探测到周跳时（可利用已有的 GF 组合探测），必须对该卫星的滤波器进行重置。平滑窗口长度 N 是一个可调参数，通常设置为 100-200 秒左右的等效长度。

3.3.2 实施加权最小二乘法 (WLS)

观测到的定位误差周期性、系统性偏差以及对数据质量变化的敏感性，主要归因于采用了未加权的最小二乘估计算法，该算法未能有效区分和处理不同质量的卫星观测值。为提升该导航定位软件的性能，可以实施加权最小二乘法（WLS），根据卫星高度角或载噪比为观测值赋予权重，这将直接改善定位精度并减少系统偏差。

将最小二乘解的核心计算修改为 $\Delta x=(B^TPB)^{-1}B^TPL$ ，关键在于构建对角权重矩阵 P。

（1）基于高度角的模型： 一个常用的模型将第 i 颗卫星观测值的方差设置为 $\sigma_i^2=\dfrac{\sigma_0^2}{\sin^2(El_i)}$ ，其中 El_i 是卫星的高度角。相应的权重为 $P_i=\dfrac{1}{\sigma_i^2}=\dfrac{\sin^2(El_i)}{\sigma_0^2}$ ，由于高度角在程序中已有计算 ，此模型实现简单。

（2）基于载噪比(C/N_0)的模型：一个更有效的模型是利用载噪比（C/N_0），它是衡量信号质量的直接指标，并已在 SATOBSDATA 结构体中提供（cn）。观测

值方差可以建模为与 C/N_0成反比，例如$\sigma_i^2 = a + b \cdot 10^{-0.1 \cdot (C/N_{0i})}$ ，该模型能更灵敏地反映真实的信号传播条件，包括多路径等导致的 C/N_0下降 。模型参数 a 和 b 可以通过经验数据进行调整。

3.3.3 卡尔曼滤波器 (KF)

当然，逐历元的最小二乘法虽然有效，但它忽略了接收机状态在时间上的相关性。接收机的位置和速度不会在每秒之间随机跳变。卡尔曼滤波器利用一个动态模型（如匀速模型）来预测下一时刻的状态，然后用新的观测值来更新这个预测。这会产生一个更平滑、更连续，并且通常更精确的解，尤其是在速度估计方面 。在卫星数量暂时少于 4 颗的情况下，KF 也比 LS 更为稳健。

四、心得与体会

为学之实，固在践履，《卫星导航算法与程序设计》这门课程为我架设了一个从理论知识到实践的桥梁。在这四个月的课程学习中，我不断在加深对算法的理解、不断在思考对函数的设计、不断在学习编程的技巧，最终不仅完成了程序编写与功能实现，更在这一过程中实现了知识与编程能力的双重成长。

算法的编程实现，让理论知识从书本上的文字转化为可感知的逻辑，极大深化了我的理解。此前《卫星导航原理》课程中，卫星位置计算、投影函数编写、观测值线性组合等实习作业，虽为我奠定了编程实践的基础，但知识点仍呈零散状态。而本课程通过单点定位的完整实现，将卫星导航原理的知识体系编织成了有机整体：从二进制文件或串口网络获取数据时，我既厘清了文件与串口模式的差异，也掌握了二进制数据格式的解析逻辑；对数据进行周跳与粗差探测、观测值线性组合后，再进行卫星位置解算 —— 其中涵盖卫星位置、速度、钟差、钟速的计算，以及相对论效应、地球自转、对流层延迟等改正项的实现；最终结合最小二乘原理完成单点定位（SPP）与单点测速（SPV）解算，并输出位置信息与定位精度。这一闭环过程让知识从“点”连成“线”、织成“网”，而对理论的通透理解理解也反过来帮助我去编写和修改程序。

在完成这份程序的过程中，我还提升了编程的能力和水平。在课程开始之初，

王老师讲授了软件的流程设计和总体模块组成，让平时总是将所有结构体和函数都堆到同一个.cpp 文件下的我有了将程序进行模块化的思想。而在随后对各个模块进行结构体定义和子函数设计的过程，也更加让我体悟了模块化在增强程序可读性、可维护性、可复用性中发挥的作用—— 代码不再是杂乱的“堆砌”，而成为有序的“架构”。其次，通过这次编程，我调试代码的能力也有了很大提升。之前编程的时候，我调试代码还仅仅停留在粗糙判断是哪个函数可能有错误，这次课程中我就慢慢学会在调试代码时通过观察变量去寻找问题。调试的过程也是一个需要比较耐心、专注的过程，因为出现问题的地方往往是平时疏忽或者不是很理解的地方，比如我的 ENU 定位误差在高程方向起初出现了十米左右的误差，最后发现是 BestPos 解码时没有对 undulation 进行解码计算。虽然刚开始调试还不是很熟练，但是我觉得在这个过程中得到了很多的锻炼和成长。除此之外，向老师的请教、与同学的讨论、对 AI 的应用都让我在编写程序的过程中收获良多。

总结为期四个月的课程，我既一步步完整地实现了程序的编写，也在不断进步。我感受到整个课程的设计非常合理，围绕实现单点定位这一课程目标，王老师从整体上提供了整个程序以及各个模块的核心算法与函数设计思路，让我编写程序的时候有方向，同时王老师也给我们自己留了思考空间，让我在不断尝试中进步。我也非常喜欢这一门课程，一方面非常喜欢王老师的授课方式，有条不紊、循序渐进，另一方面这门课程也让我能用所学习到的知识去实践、创造。后续，我其实还希望能够有机会去学习优秀的代码，因为我只是尽我自己的能力去实现了程序的编写，这其中肯定有很大的进步空间，所以期待老师在所有同学都尽力实现好自己的程序后，能不能专门以一份优秀的代码作为示例来展示我们可以向哪一个方向进步。整个课程让我受益匪浅，希望今后的专业课里面有更多像这样的课程，也非常期待下个学期的高 GNSS 精度相对导航定位算法与程序设计。最后，感谢王老师的无私指导，祝您身体健康、工作顺利！